US 20250279886A1

(19) **United States**

(12) **Patent Application Publication** (10) **Pub. No.: US 2025/0279886 A1**

Kao et al. (43) **Pub. Date: Sep. 4, 2025**

(54) **PROCESSING SYSTEM AND METHOD RELATED TO ENCRYPTION**

(71) Applicant: **Wistron Corporation**, New Taipei City (TW)

(72) Inventors: **Meng-Chao Kao**, New Taipei City (TW); **Chiy Ferng Perng**, New Taipei City (TW); **Tzu-Chiang Shen**, New Taipei City (TW); **Cheng Jhih Shih**, Taipei (TW); **Shih Hao Hung**, Taipei (TW)

(73) Assignee: **Wistron Corporation**, New Taipei City (TW)

(21) Appl. No.: **18/657,718**

(22) Filed: **May 7, 2024**

(30) **Foreign Application Priority Data**

Feb. 29, 2024 (TW) 113107218

Publication Classification

(51) **Int. Cl.**

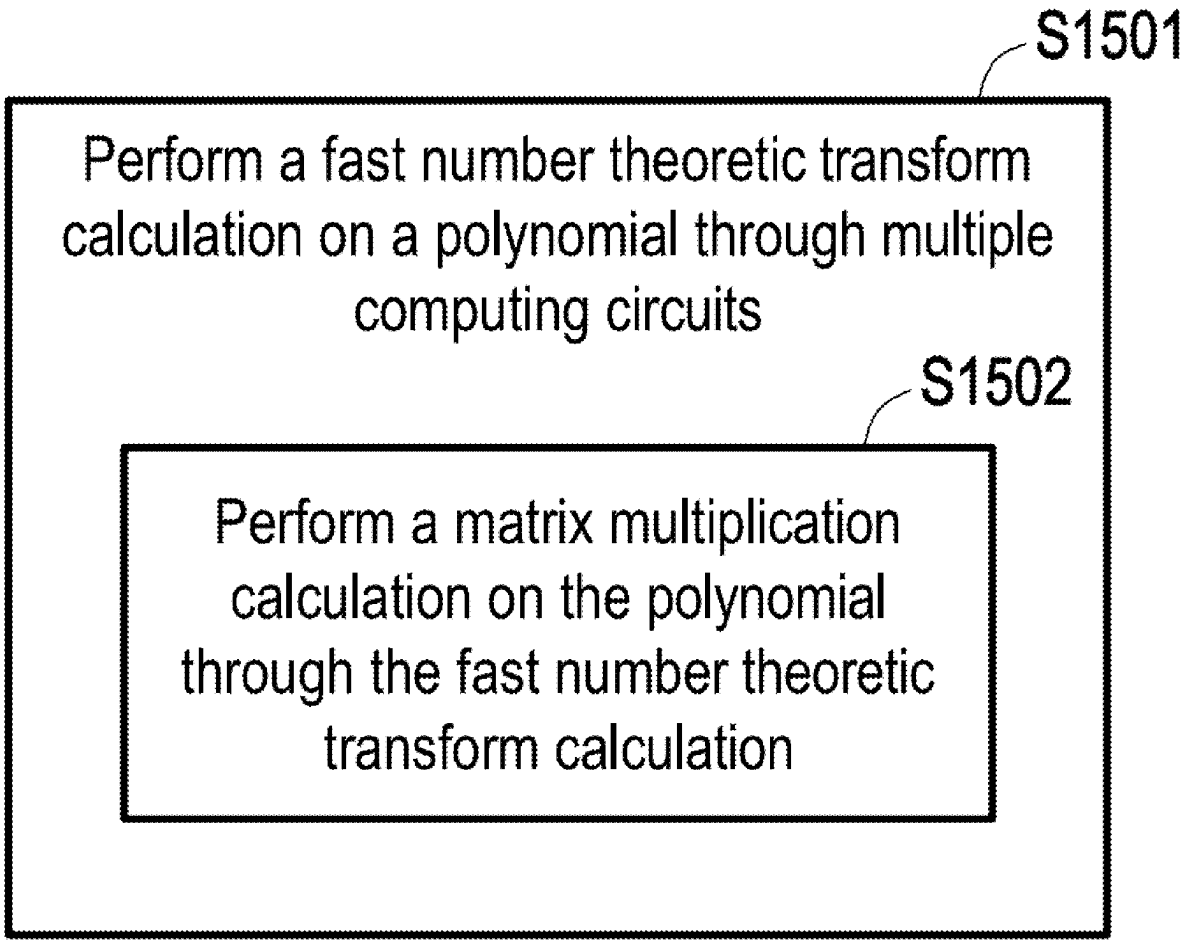
H04L 9/30	(2006.01)
G06F 17/14	(2006.01)
G06F 17/16	(2006.01)
H04L 9/00	(2022.01)

(52) **U.S. Cl.**

CPC ***H04L 9/3093*** (2013.01); ***G06F 17/142*** (2013.01); ***G06F 17/16*** (2013.01); ***H04L 9/008*** (2013.01)

(57) **ABSTRACT**

A processing system and method related to encryption are provided. The processing system includes multiple computing circuits. A memory stores data. The computing circuit performs a number theoretic transform (NTT) calculation on a polynomial. A matrix multiplication calculation is performed on the polynomial through the NTT calculation. Therefore, the computing efficiency of encryption or decryption can be improved.



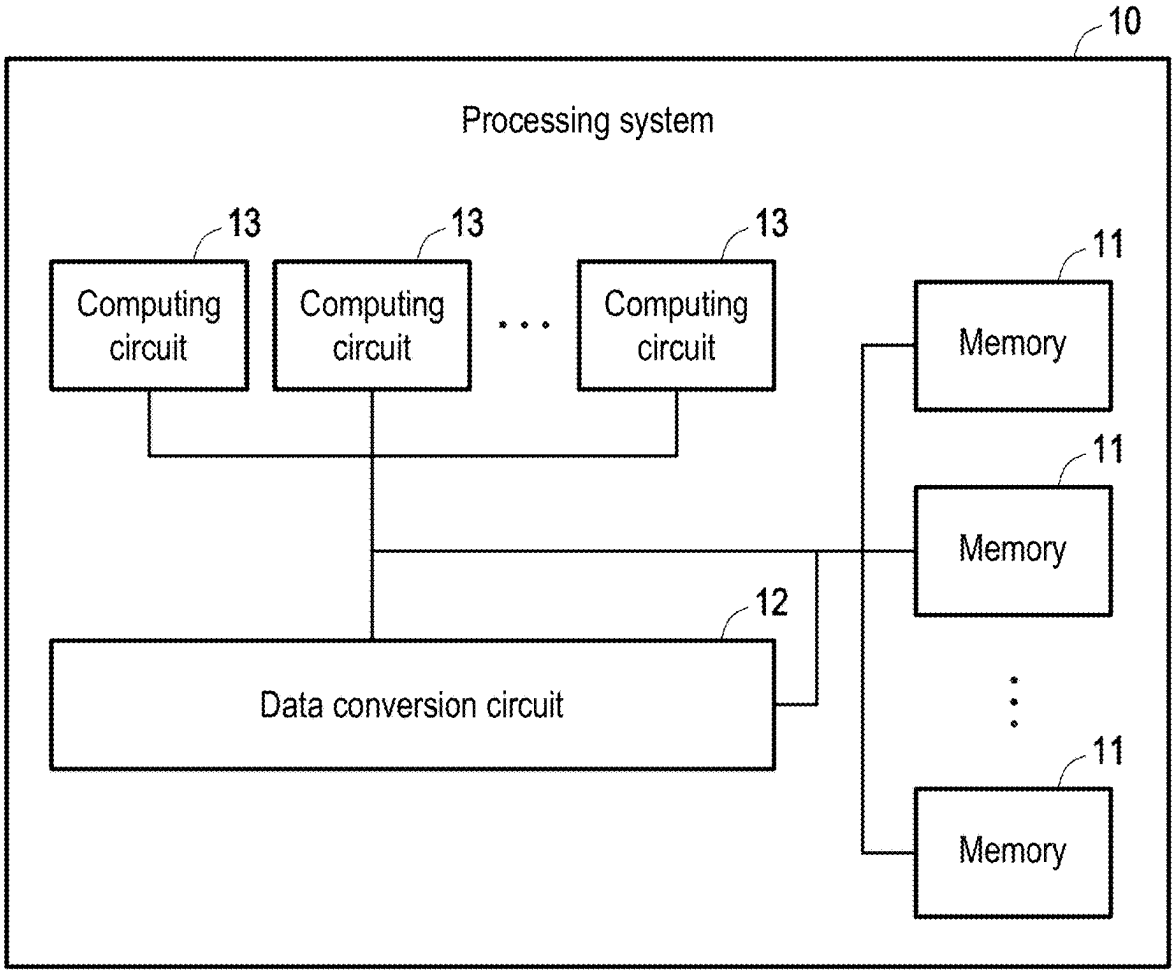

10
Processing system
13
13
13
Computing circuit
Computing circuit
Computing circuit
11
Memory
11
Memory
12
Data conversion circuit
11
Memory

FIG. 1

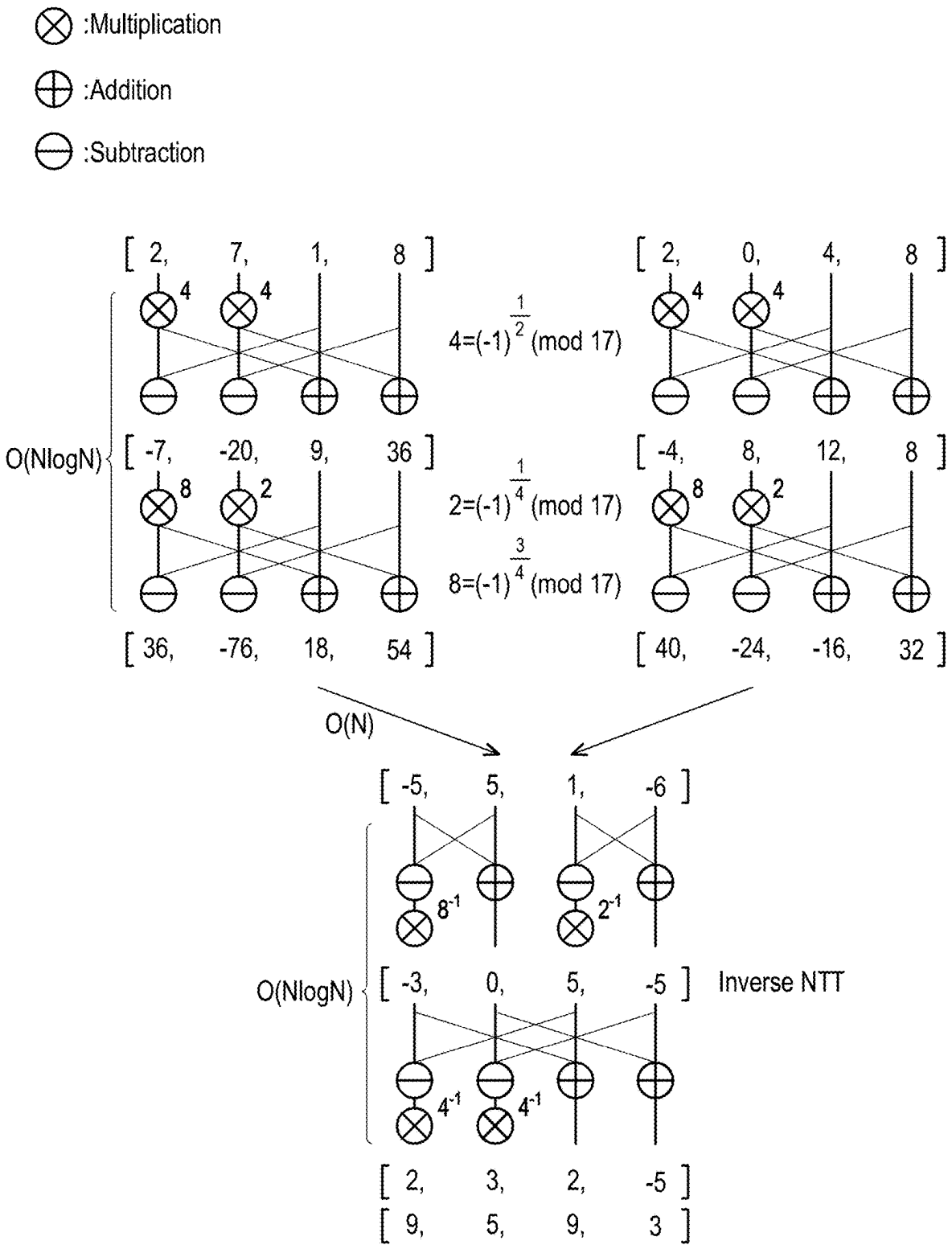

:Multiplication
:Addition
:Subtraction
[2, 7, 1, 8]
[2, 0, 4, 8]
4=(-1)^(1/2) (mod 17)
O(NlogN)
[-7, -20, 9, 36]
[-4, 8, 12, 8]
2=(-1)^(1/4) (mod 17)
8=(-1)^(3/4) (mod 17)
[36, -76, 18, 54]
[40, -24, -16, 32]
O(N)
[-5, 5, 1, -6]
O(NlogN)
[-3, 0, 5, -5]
Inverse NTT
[2, 3, 2, -5]
[9, 5, 9, 3]

FIG. 2

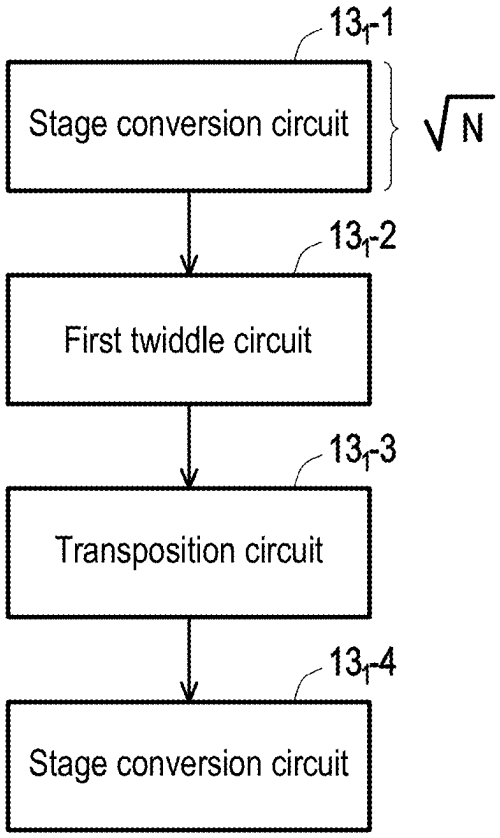

13_1-1
Stage conversion circuit
$\sqrt{N}$
13_1-2
First twiddle circuit
13_1-3
Transposition circuit
13_1-4
Stage conversion circuit

FIG. 3

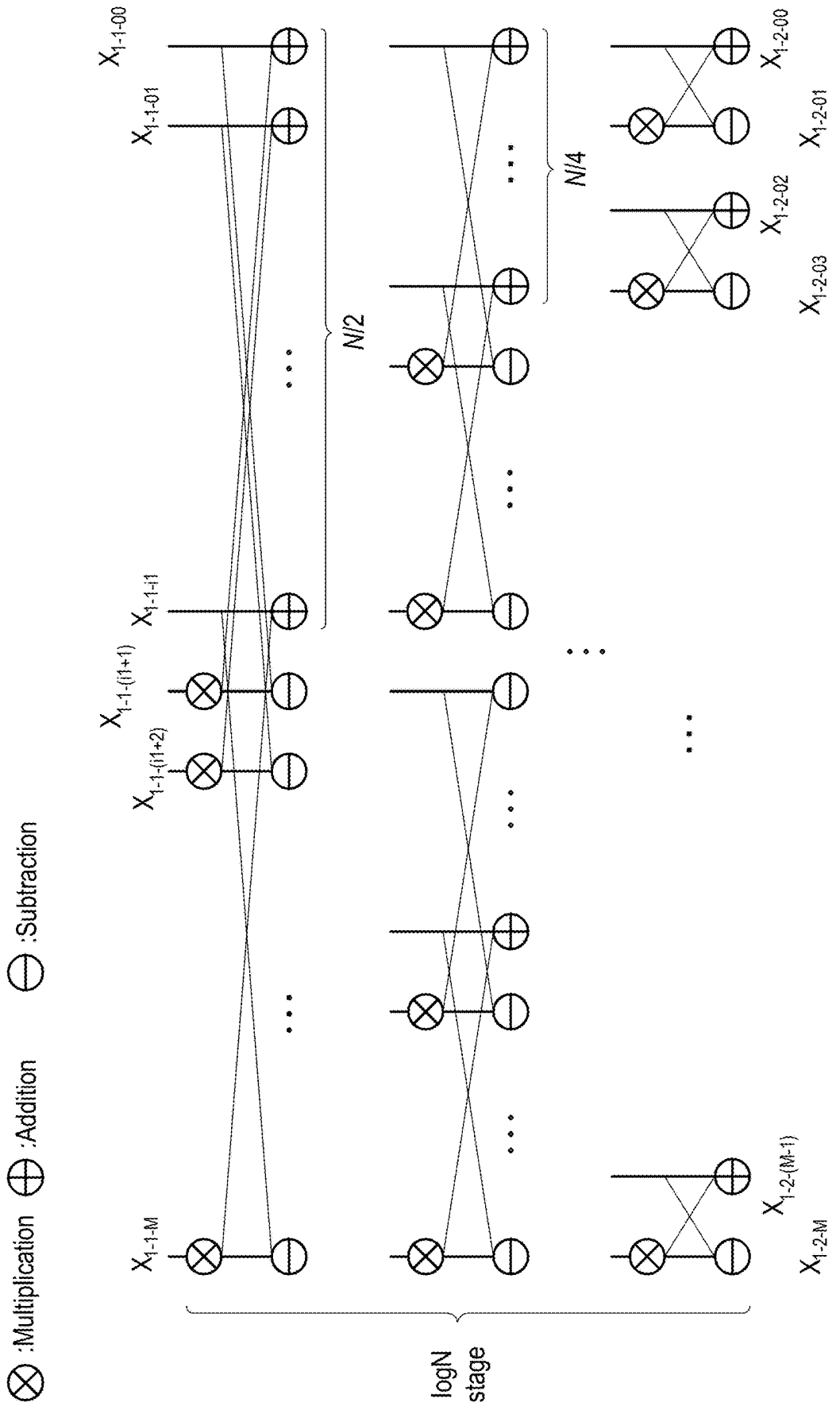

⊗ :Multiplication
⊕ :Addition
⊖ :Subtraction
X1-1-00
X1-1-01
X1-1-i1
X1-1-(i1+1)
X1-1-(i1+2)
X1-1-M
N/2
N/4
X1-2-00
X1-2-01
X1-2-02
X1-2-03
X1-2-(M-1)
X1-2-M
logN stage

FIG. 4A

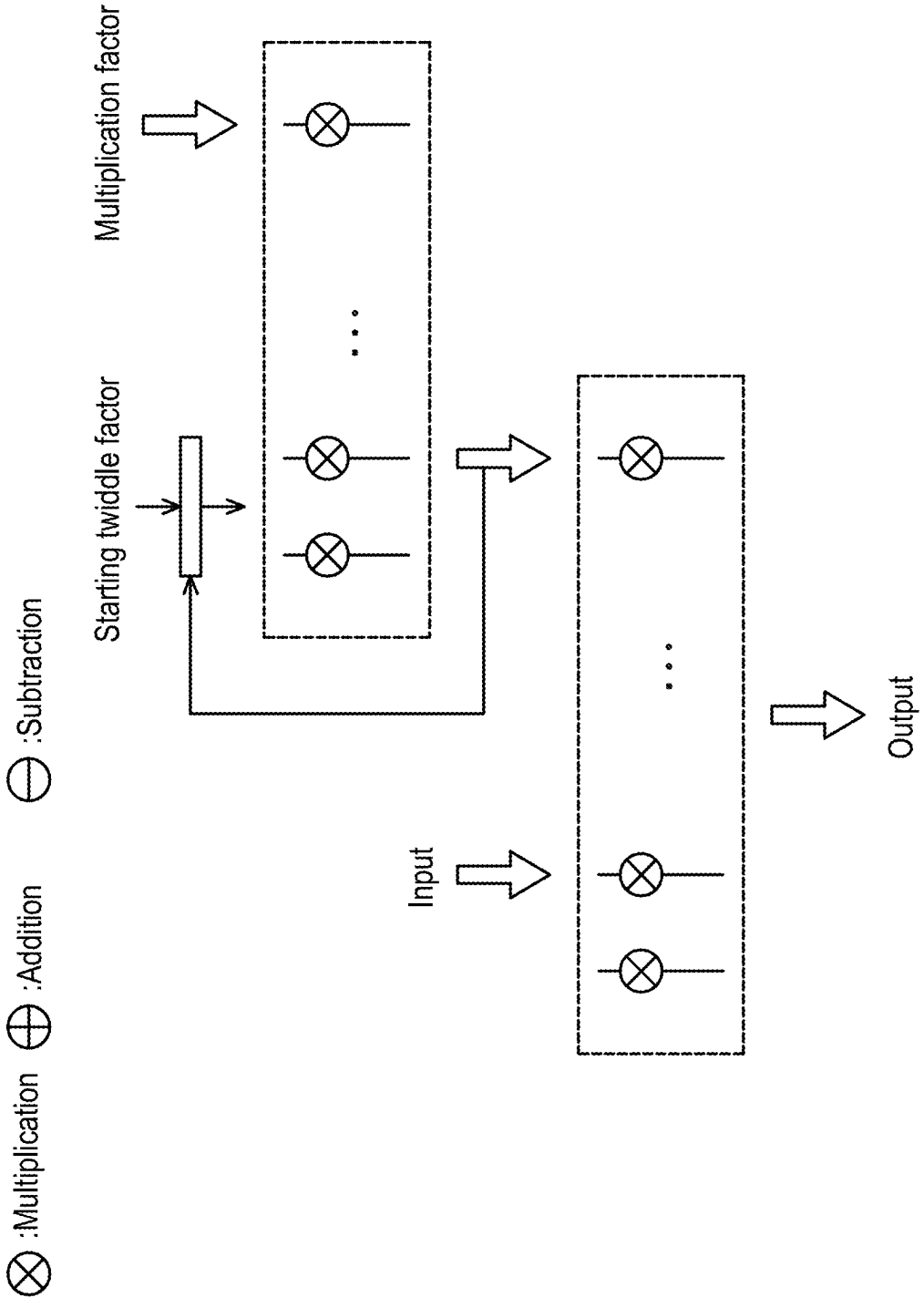

:Multiplication
:Addition
:Subtraction
Multiplication factor
Starting twiddle factor
Input
Output

FIG. 4B

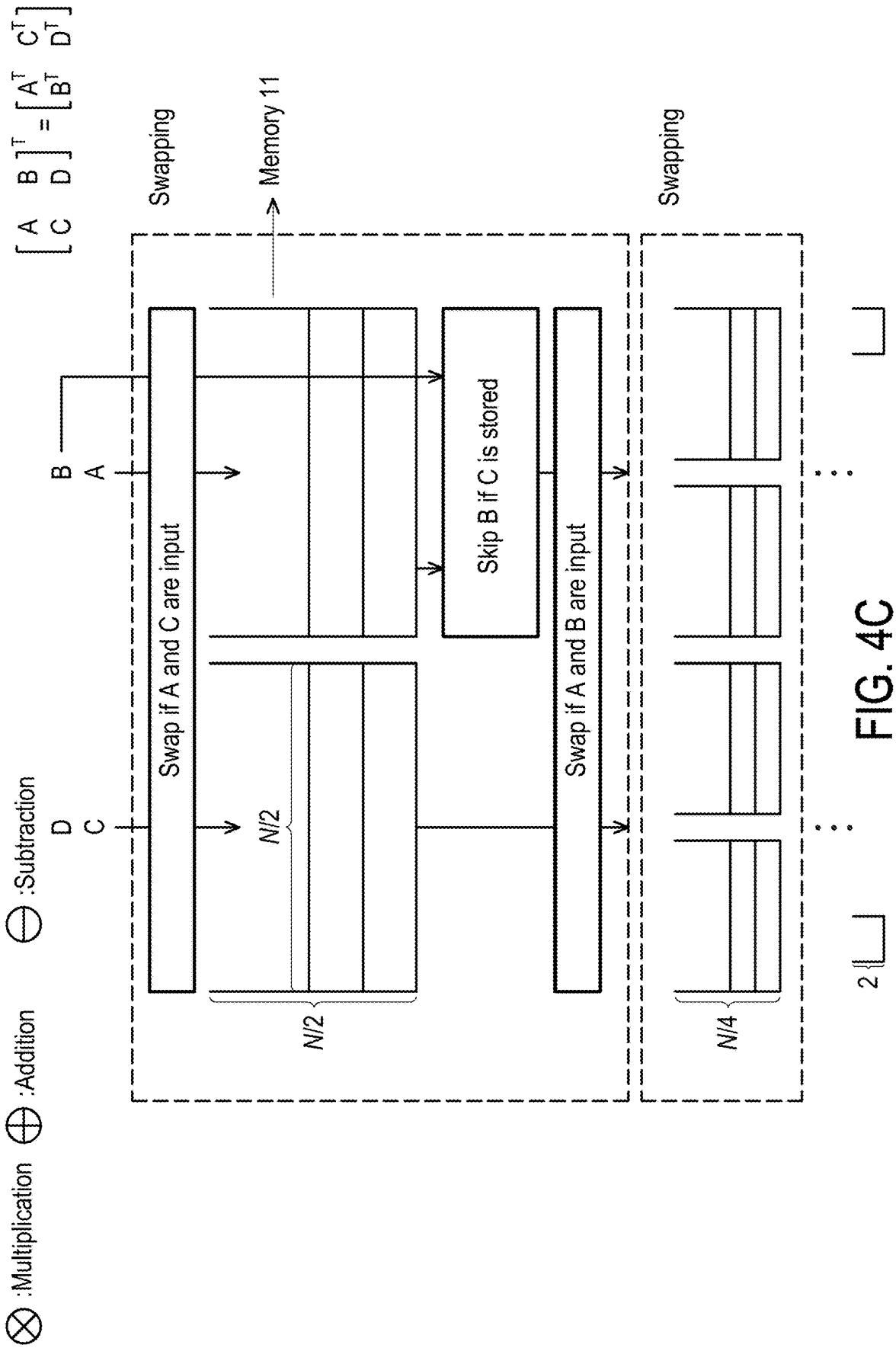

⊗ :Multiplication
⊕ :Addition
⊖ :Subtraction
$\begin{bmatrix} A & B \\ C & D \end{bmatrix}^T = \begin{bmatrix} A^T & C^T \\ B^T & D^T \end{bmatrix}$
Swapping
Memory 11
Swapping
B
A
D
C
Swap if A and C are input
Skip B if C is stored
Swap if A and B are input
N/2
N/2
N/4
2

FIG. 4C

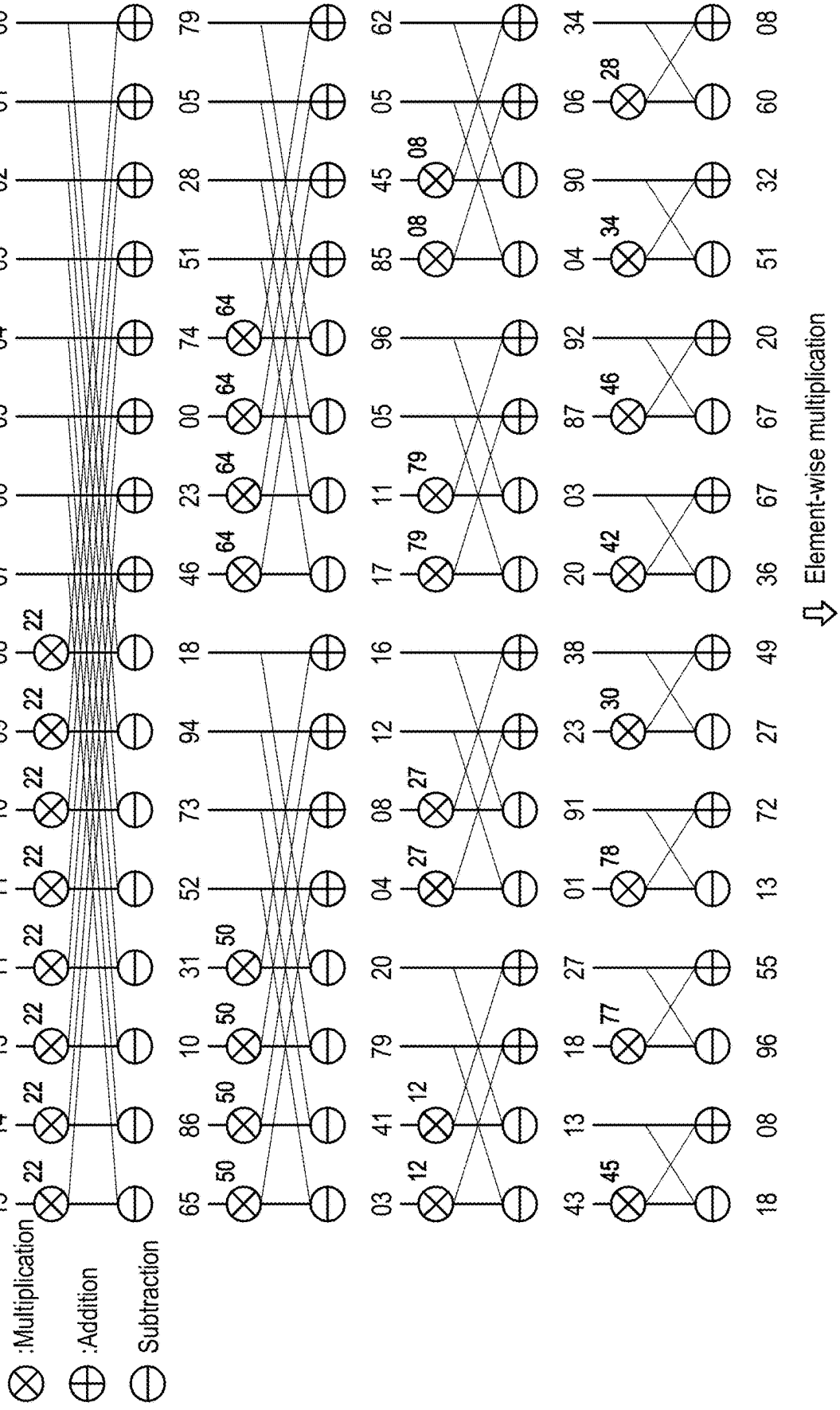

:Multiplication
:Addition
Subtraction
Element-wise multiplication

FIG. 5A

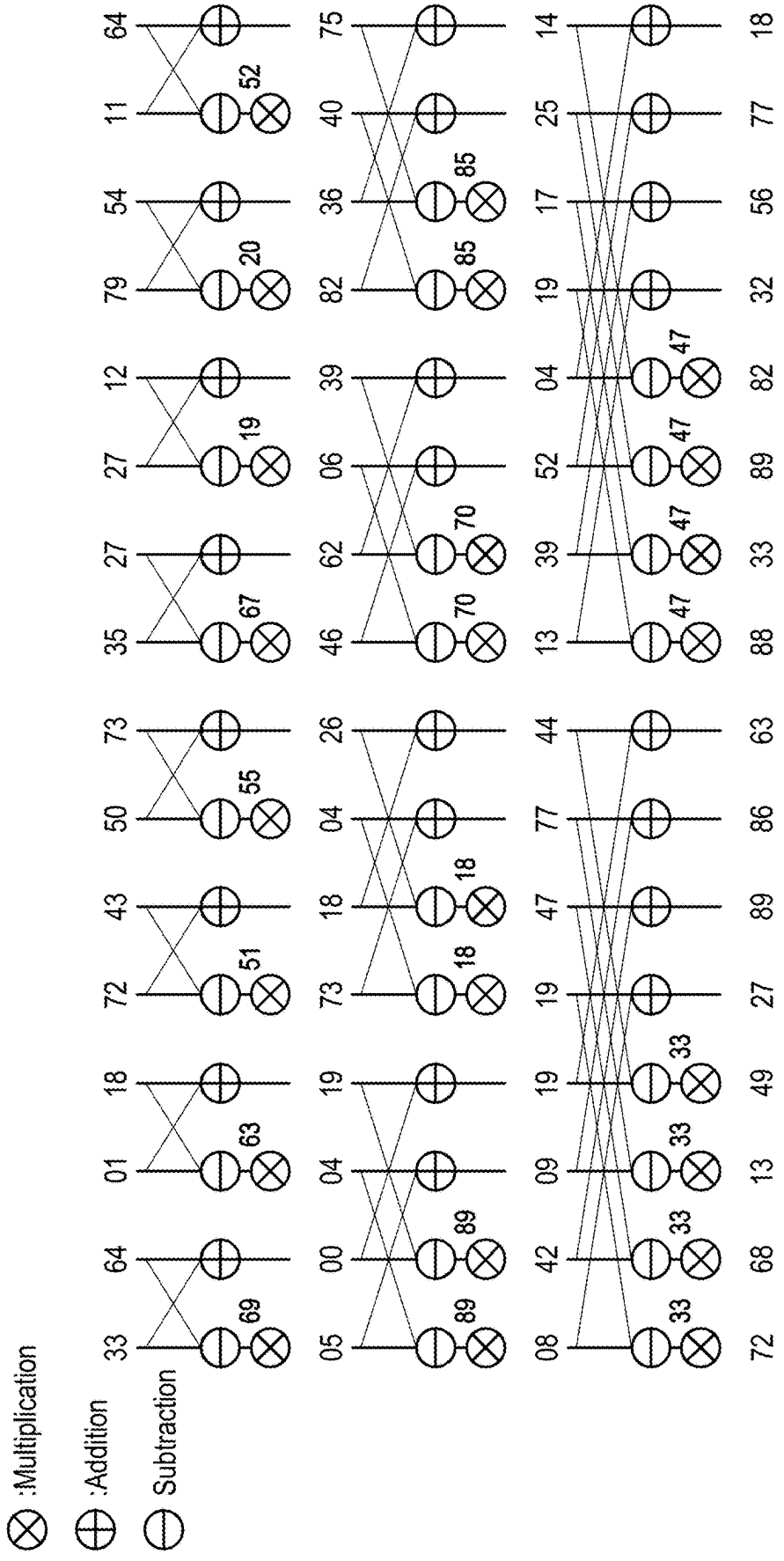

:Multiplication
:Addition
Subtraction
33 64 01 18 72 43 50 73 35 27 27 12 79 54 11 64
05 00 04 19 73 18 04 26 46 62 06 39 82 36 40 75
08 42 09 19 19 47 77 44 13 39 52 04 19 17 25 14
72 68 13 49 27 89 86 63 88 33 89 82 32 56 77 18
69 63 51 55 67 19 20 52
89 89 18 18 70 70 85 85
33 33 33 33 47 47 47 47

FIG. 5B

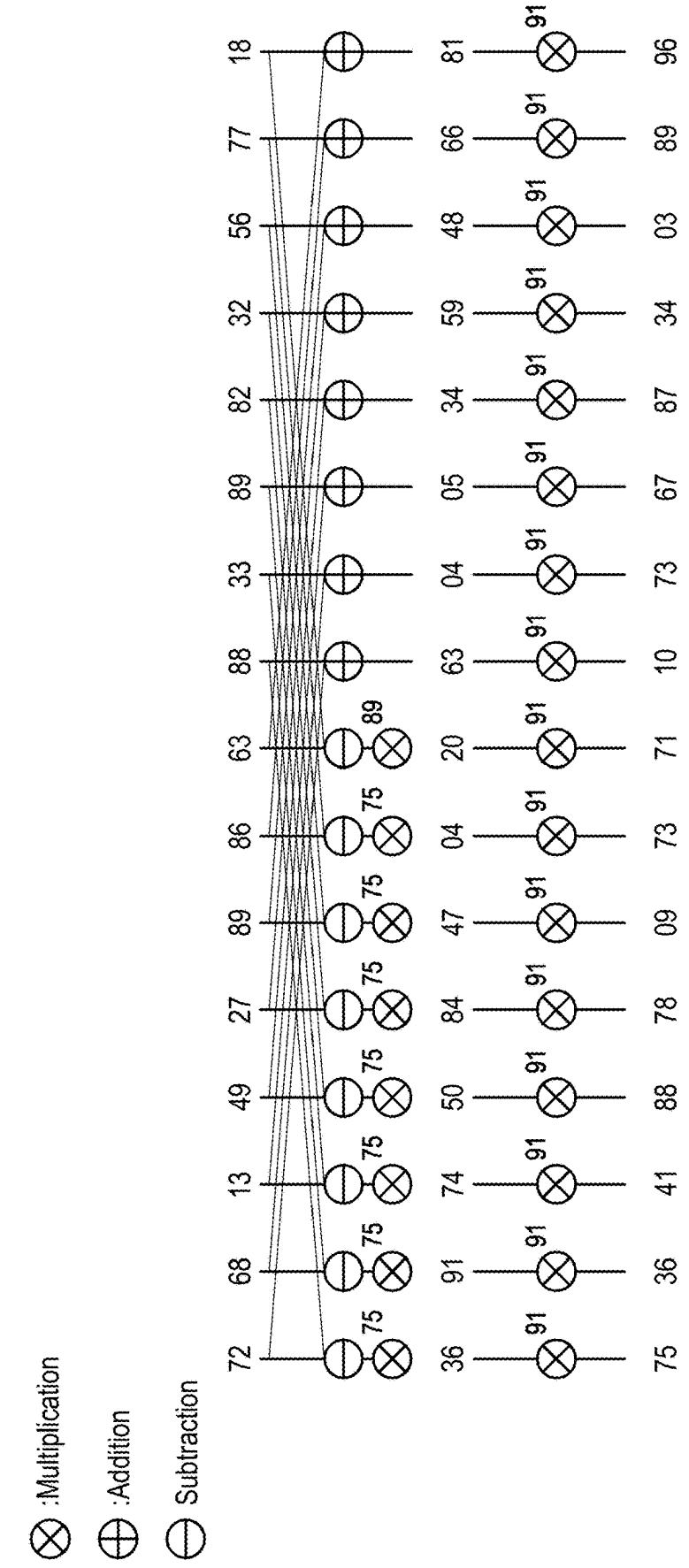

:Multiplication
:Addition
Subtraction
18 77 56 32 82 89 33 88 63 86 89 27 49 13 68 72
81 66 48 59 34 05 04 63 20 04 47 84 50 74 91 36
96 89 03 34 87 67 73 10 71 73 09 78 88 41 36 75
91 91 91 91 91 91 91 91 89 75 75 75 75 75 75 75
91 91 91 91 91 91 91 91 91 91 91 91 91 91 91 91

FIG. 5C

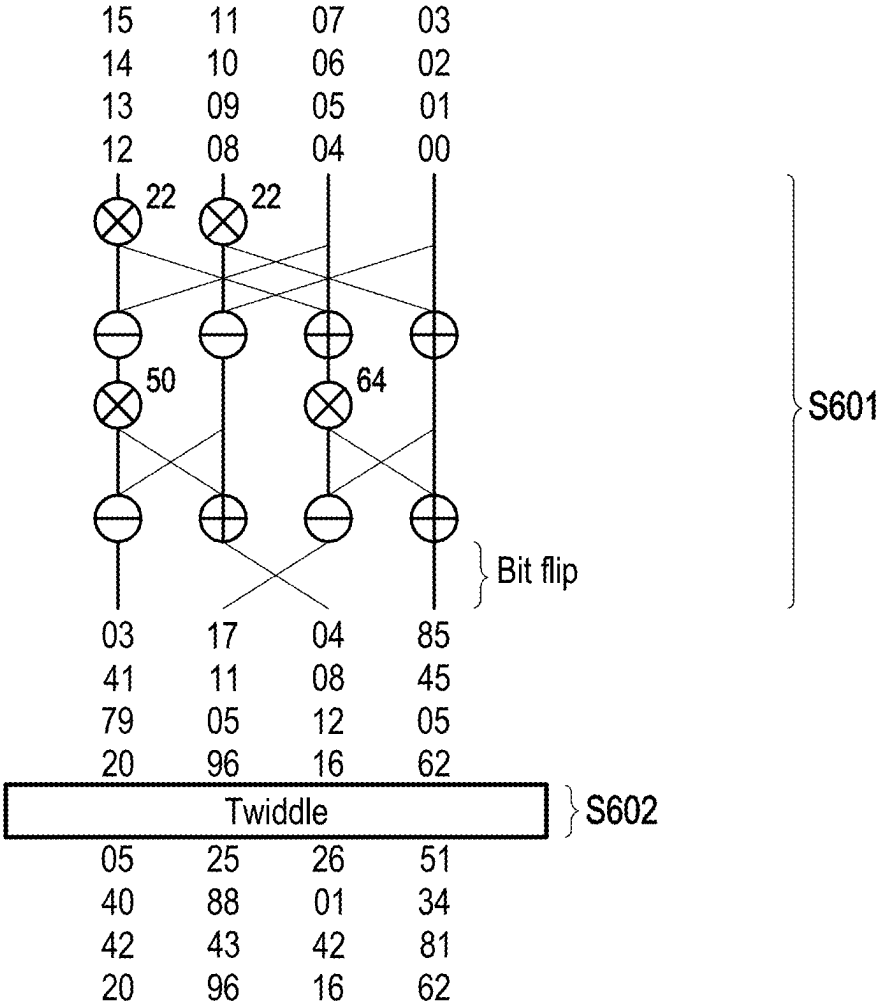

15 11 07 03
14 10 06 02
13 09 05 01
12 08 04 00
22
22
50
64
S601
Bit flip
03 17 04 85
41 11 08 45
79 05 12 05
20 96 16 62
Twiddle
S602
05 25 26 51
40 88 01 34
42 43 42 81
20 96 16 62

FIG. 6A

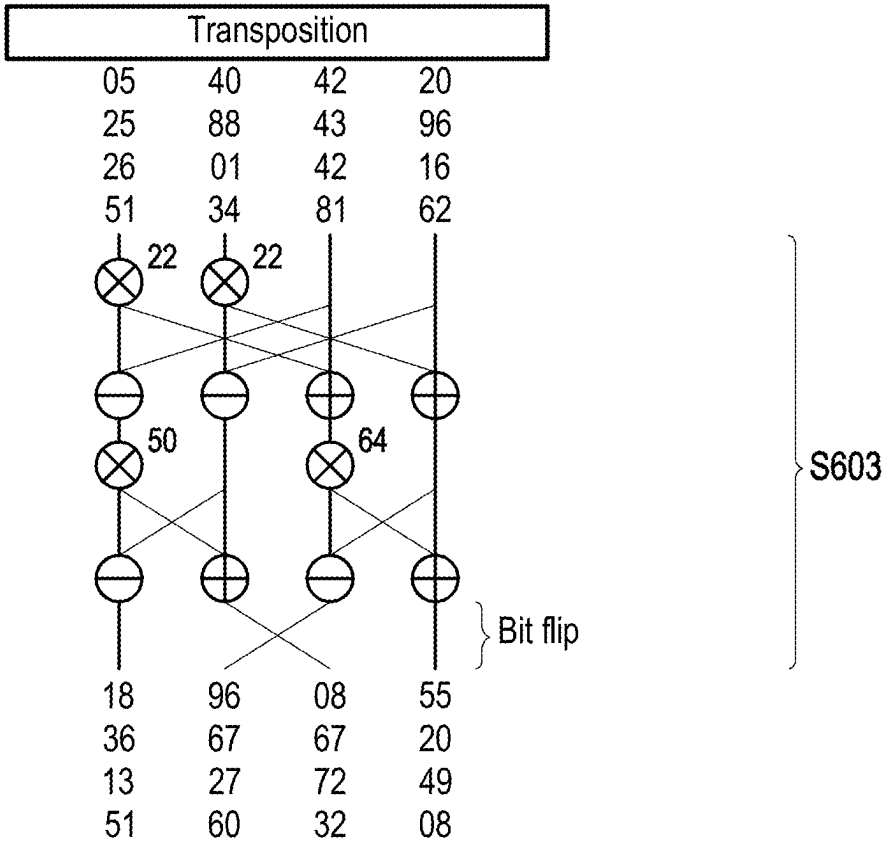

Transposition
05 40 42 20
25 88 43 96
26 01 42 16
51 34 81 62
22
22
50
64
Bit flip
S603
18 96 08 55
36 67 67 20
13 27 72 49
51 60 32 08

FIG. 6B

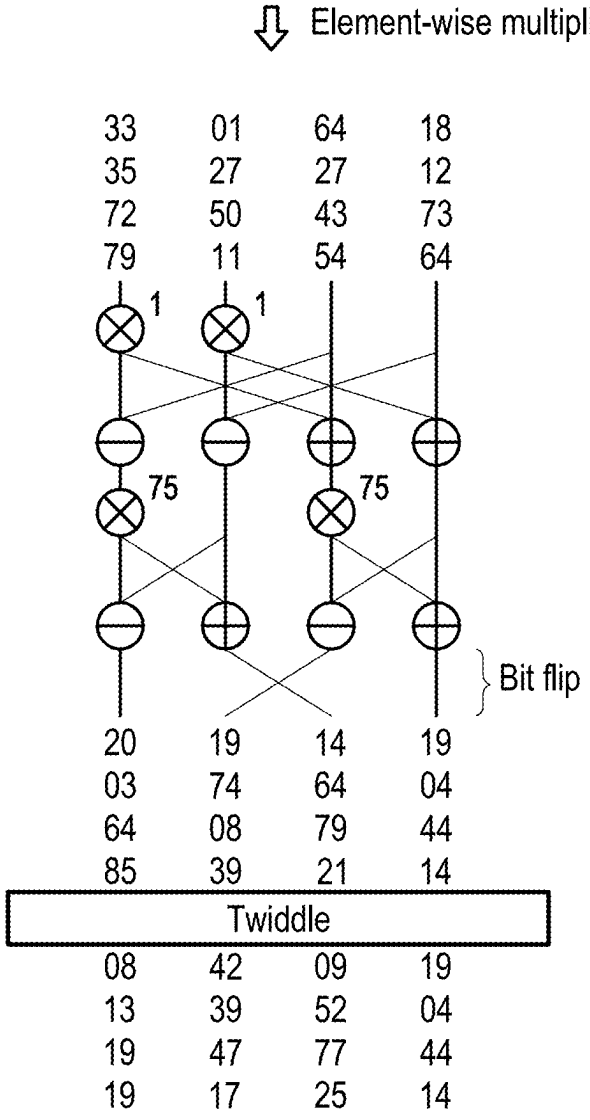

Element-wise multiplication
33 01 64 18
35 27 27 12
72 50 43 73
79 11 54 64
1
1
75
75
Bit flip
20 19 14 19
03 74 64 04
64 08 79 44
85 39 21 14
Twiddle
08 42 09 19
13 39 52 04
19 47 77 44
19 17 25 14

FIG. 6C

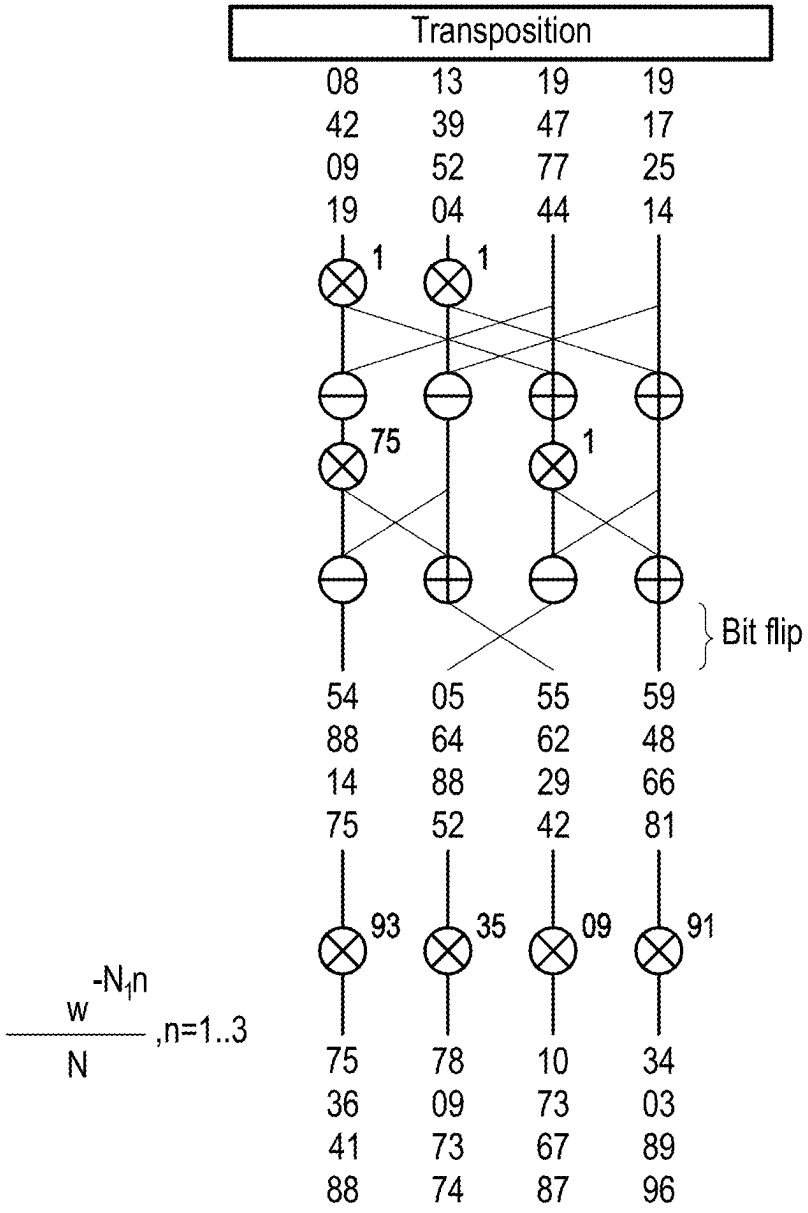

Transposition
08 13 19 19
42 39 47 17
09 52 77 25
19 04 44 14
1
1
75
1
Bit flip
54 05 55 59
88 64 62 48
14 88 29 66
75 52 42 81
93
35
09
91
$\frac{w^{-N_1 n}}{N}$,n=1..3
75 78 10 34
36 09 73 03
41 73 67 89
88 74 87 96

FIG. 6D

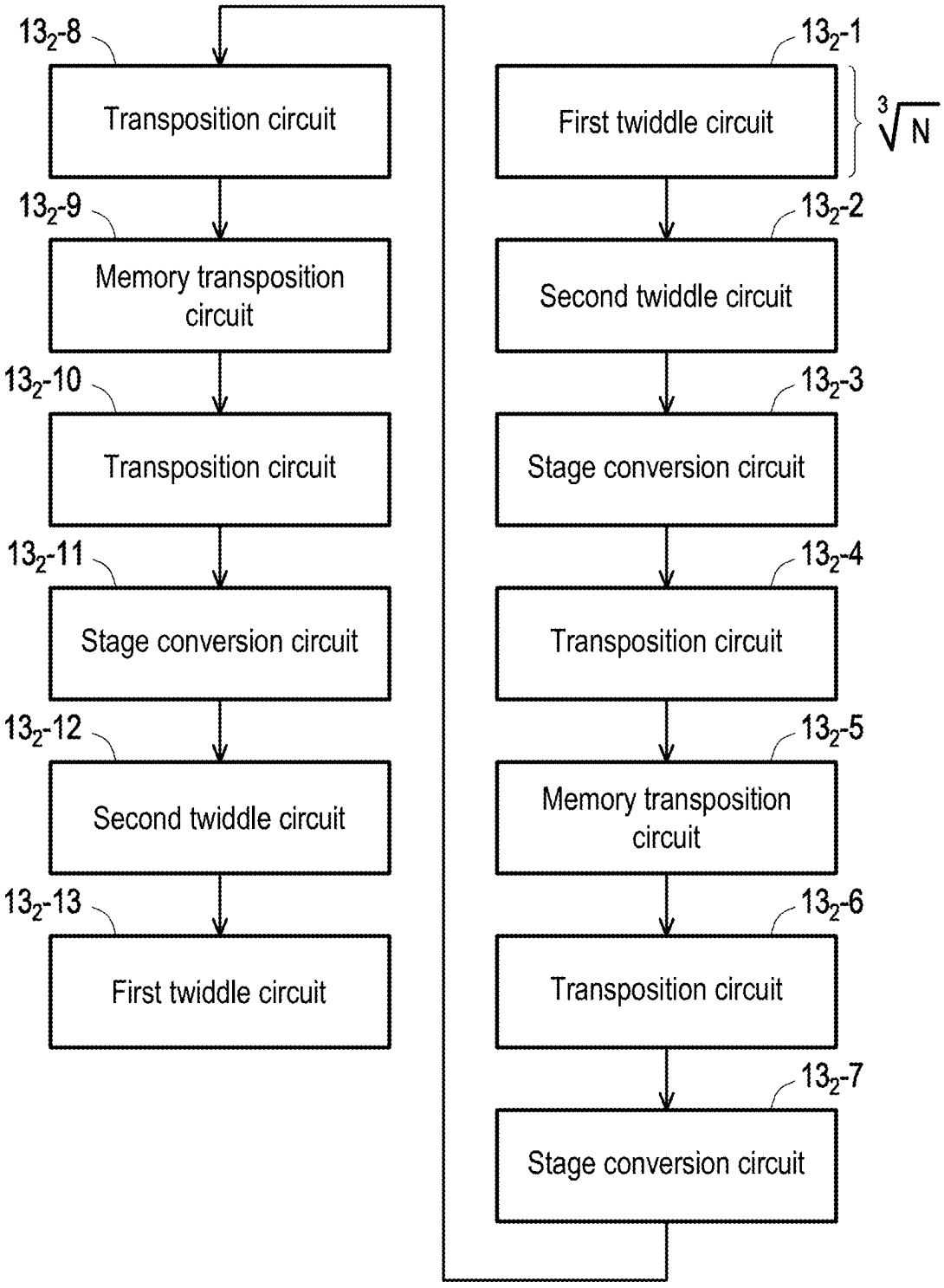

13_2-8
Transposition circuit
13_2-9
Memory transposition circuit
13_2-10
Transposition circuit
13_2-11
Stage conversion circuit
13_2-12
Second twiddle circuit
13_2-13
First twiddle circuit
13_2-1
First twiddle circuit
$\sqrt[3]{N}$
13_2-2
Second twiddle circuit
13_2-3
Stage conversion circuit
13_2-4
Transposition circuit
13_2-5
Memory transposition circuit
13_2-6
Transposition circuit
13_2-7
Stage conversion circuit

FIG. 7

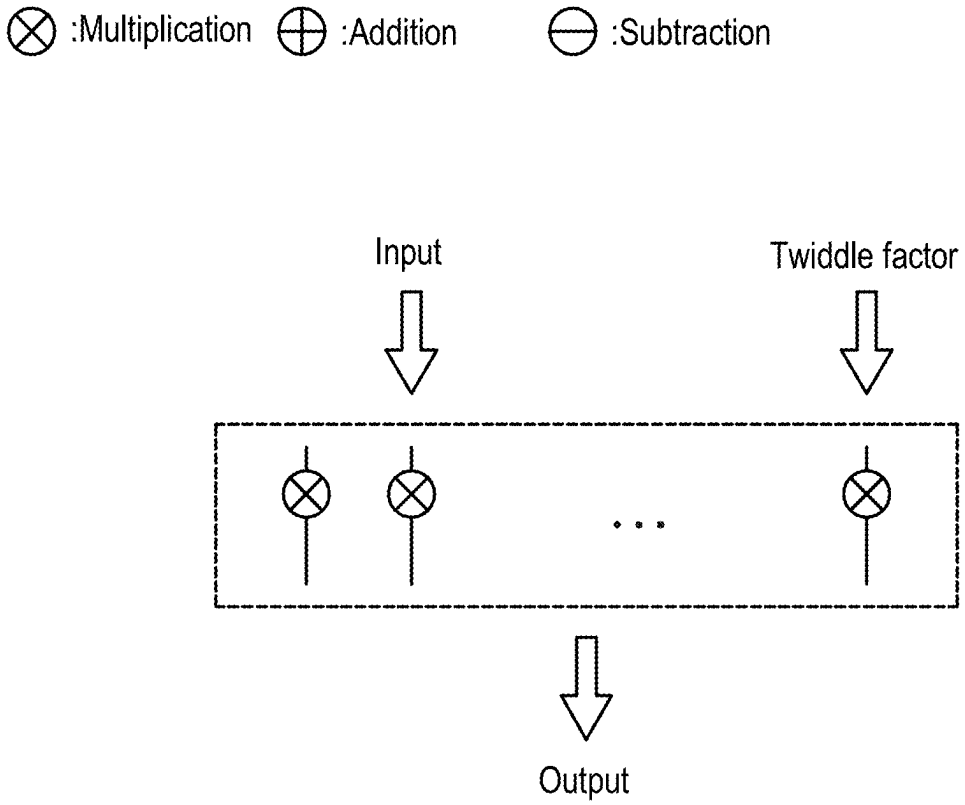

:Multiplication
:Addition
:Subtraction
Input
Twiddle factor
Output

FIG. 8A

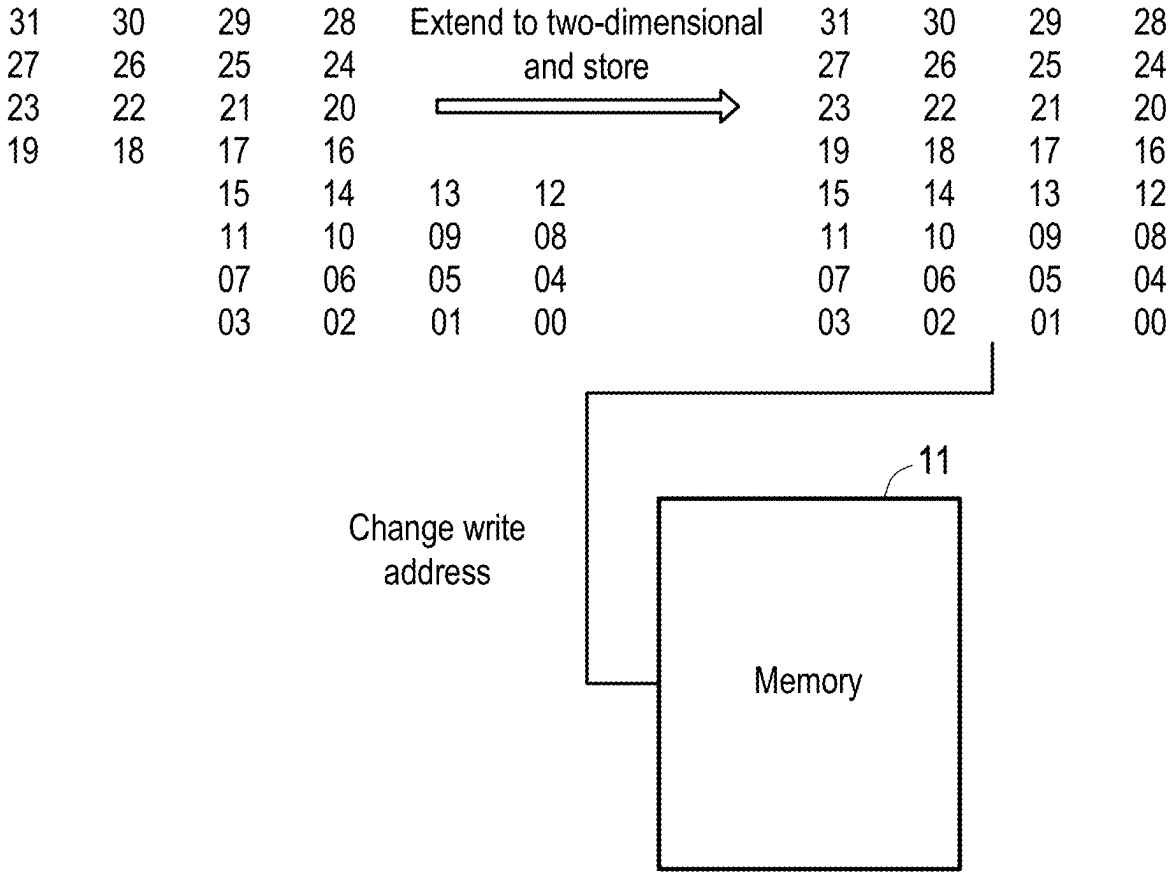

31 30 29 28
27 26 25 24
23 22 21 20
19 18 17 16
15 14 13 12
11 10 09 08
07 06 05 04
03 02 01 00
Extend to two-dimensional and store
31 30 29 28
27 26 25 24
23 22 21 20
19 18 17 16
15 14 13 12
11 10 09 08
07 06 05 04
03 02 01 00
11
Change write address
Memory

FIG. 8B

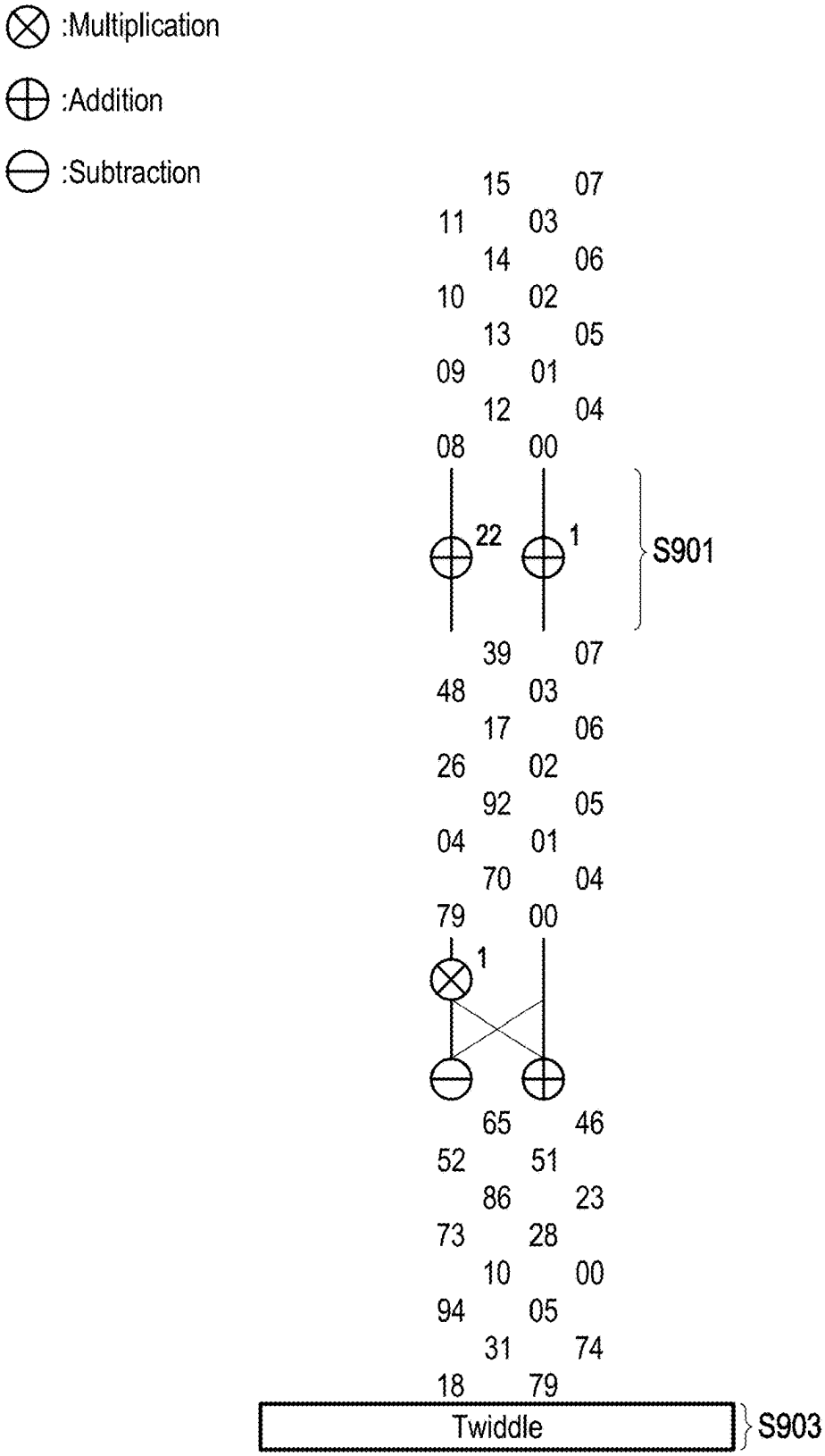

:Multiplication
:Addition
:Subtraction
15 07
11 03
14 06
10 02
13 05
09 01
12 04
08 00
22
1
S901
39 07
48 03
17 06
26 02
92 05
04 01
70 04
79 00
1
65 46
52 51
86 23
73 28
10 00
94 05
31 74
18 79
Twiddle
S903

FIG. 9A

⊗ :Multiplication

⊕ :Addition

⊖ :Subtraction

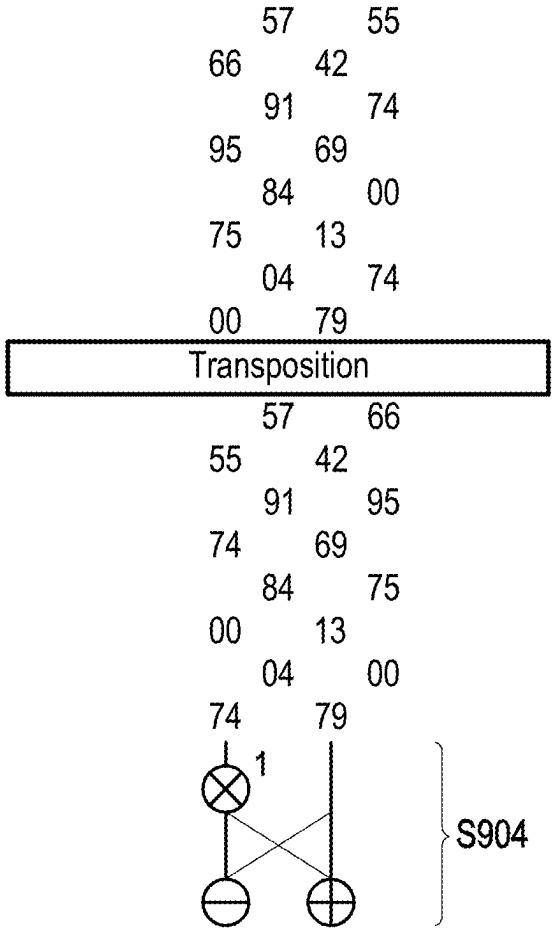


FIG. 9B

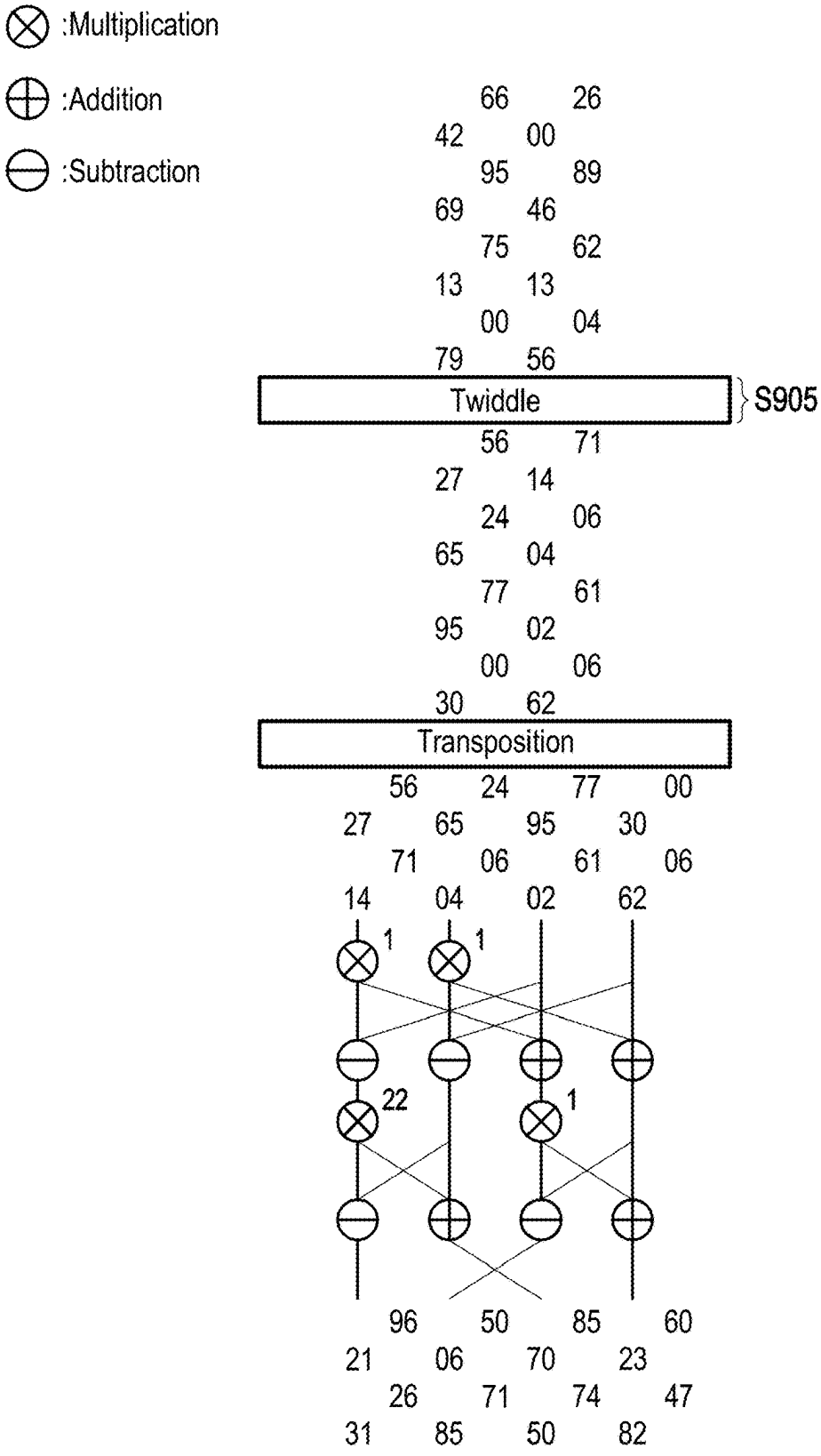

:Multiplication
:Addition
:Subtraction
66 26
42 00
95 89
69 46
75 62
13 13
00 04
79 56
Twiddle
S905
56 71
27 14
24 06
65 04
77 61
95 02
00 06
30 62
Transposition
56 24 77 00
27 65 95 30
71 06 61 06
14 04 02 62
1
1
22
1
96 50 85 60
21 06 70 23
26 71 74 47
31 85 50 82

FIG. 9C

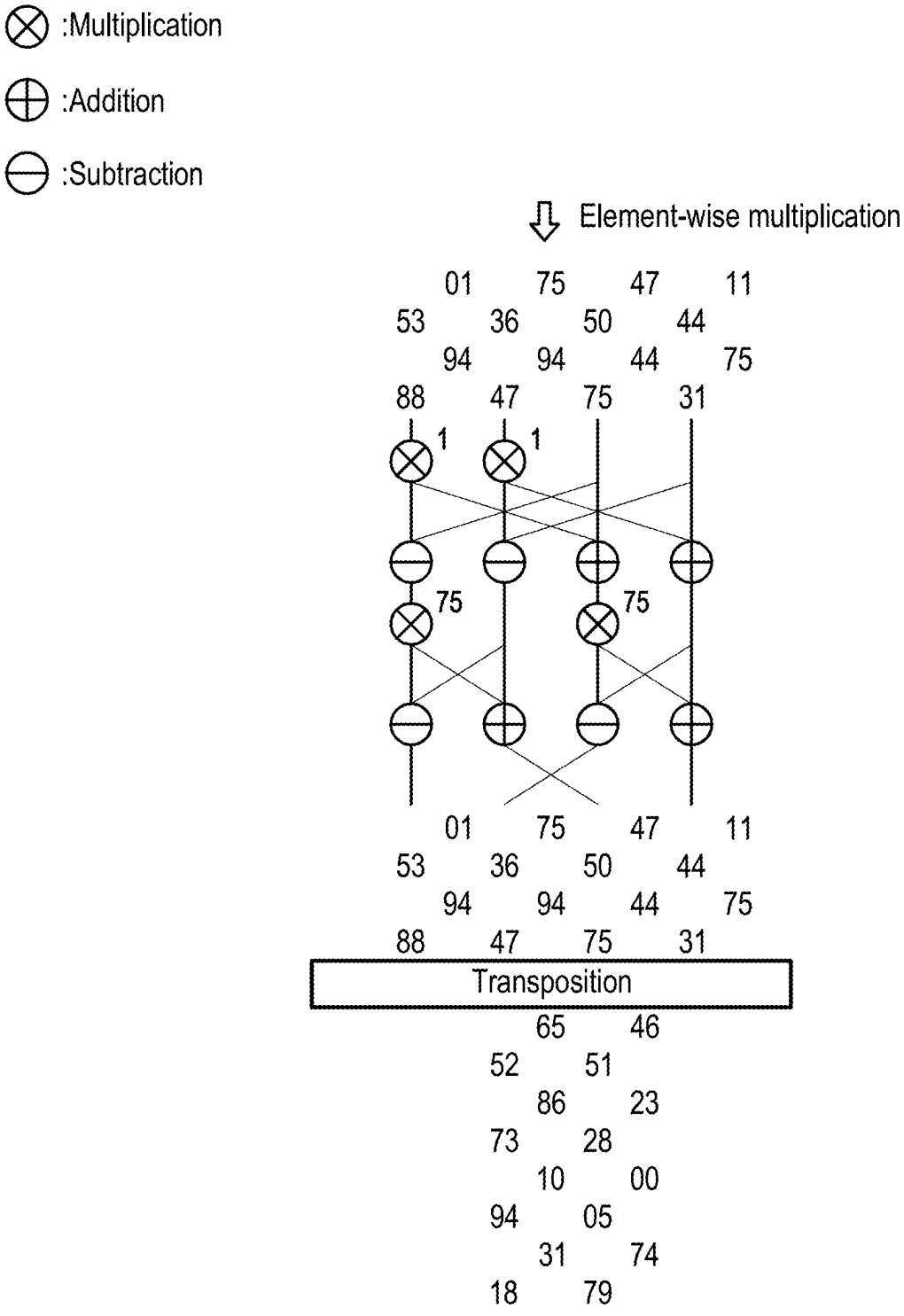

:Multiplication
:Addition
:Subtraction
Element-wise multiplication
01 75 47 11
53 36 50 44
94 94 44 75
88 47 75 31
1
1
75
75
01 75 47 11
53 36 50 44
94 94 44 75
88 47 75 31
Transposition
65 46
52 51
86 23
73 28
10 00
94 05
31 74
18 79

FIG. 9D

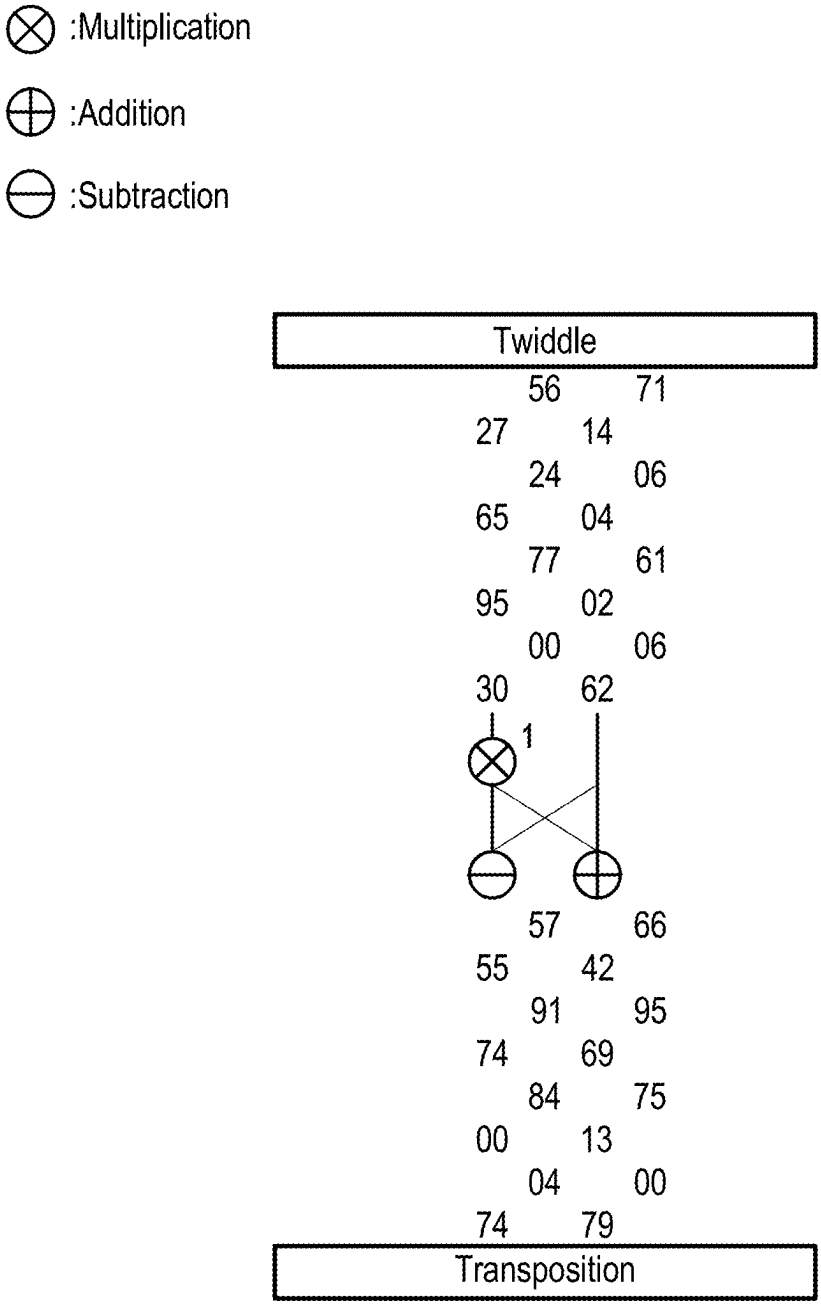

:Multiplication
:Addition
:Subtraction
Twiddle
56 71
27 14
24 06
65 04
77 61
95 02
00 06
30 62
1
57 66
55 42
91 95
74 69
84 75
00 13
04 00
74 79
Transposition

FIG. 9E

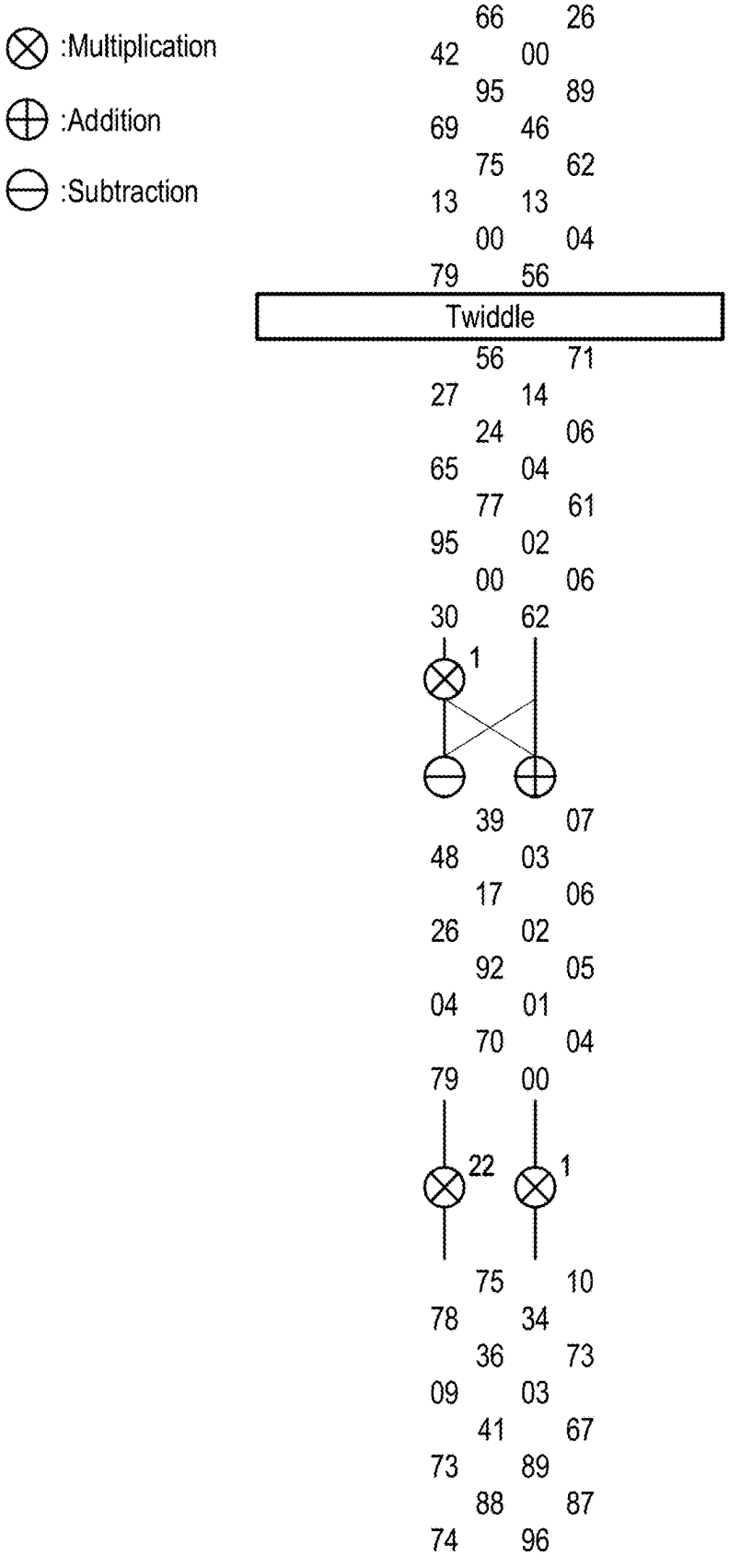

:Multiplication
:Addition
:Subtraction
66 26
42 00
95 89
69 46
75 62
13 13
00 04
79 56
Twiddle
56 71
27 14
24 06
65 04
77 61
95 02
00 06
30 62
1
39 07
48 03
17 06
26 02
92 05
04 01
70 04
79 00
22
1
75 10
78 34
36 73
09 03
41 67
73 89
88 87
74 96

FIG. 9F

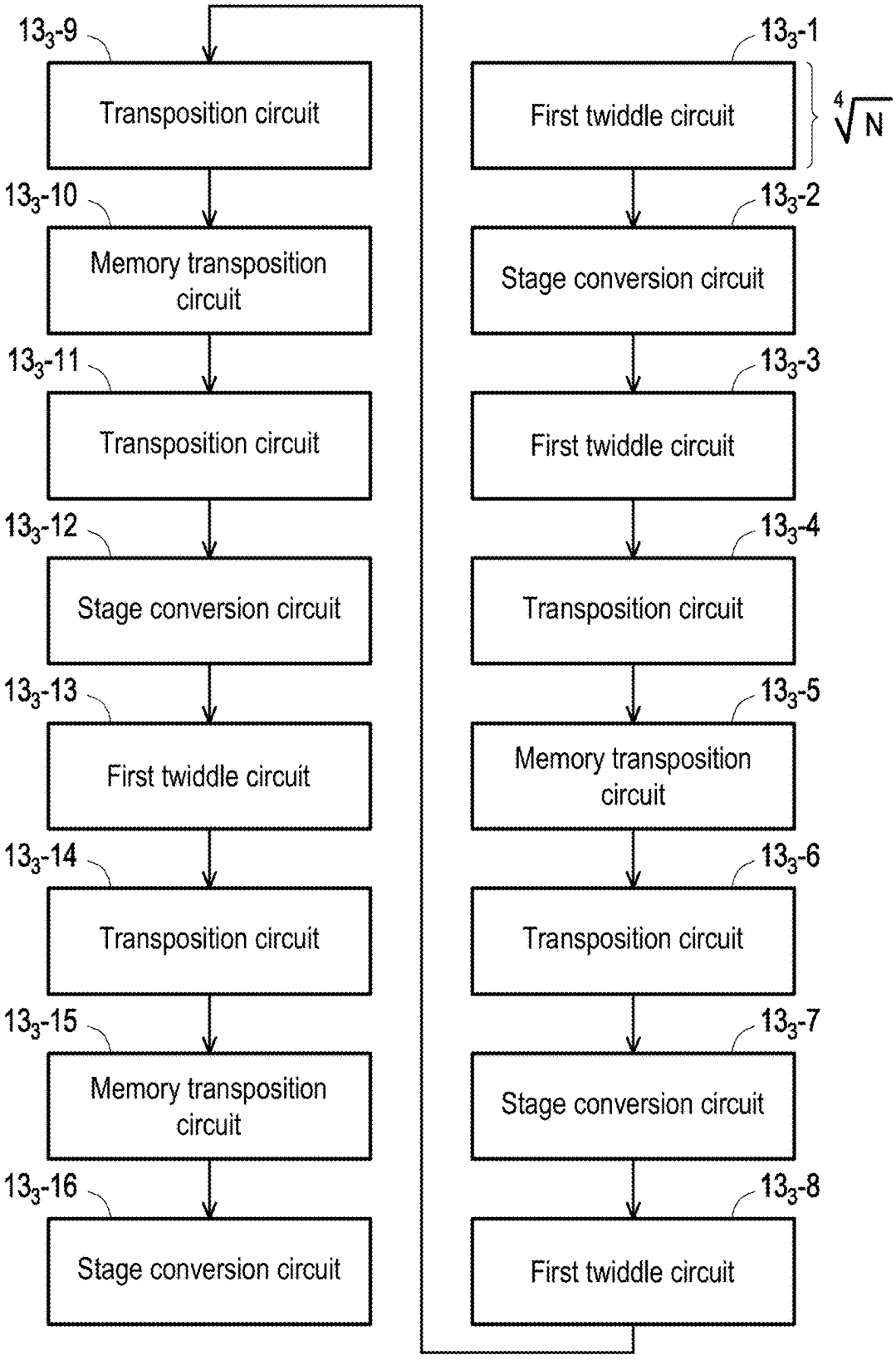

13_3-9
Transposition circuit
13_3-10
Memory transposition circuit
13_3-11
Transposition circuit
13_3-12
Stage conversion circuit
13_3-13
First twiddle circuit
13_3-14
Transposition circuit
13_3-15
Memory transposition circuit
13_3-16
Stage conversion circuit
13_3-1
First twiddle circuit
$\sqrt[4]{N}$
13_3-2
Stage conversion circuit
13_3-3
First twiddle circuit
13_3-4
Transposition circuit
13_3-5
Memory transposition circuit
13_3-6
Transposition circuit
13_3-7
Stage conversion circuit
13_3-8
First twiddle circuit

FIG. 10

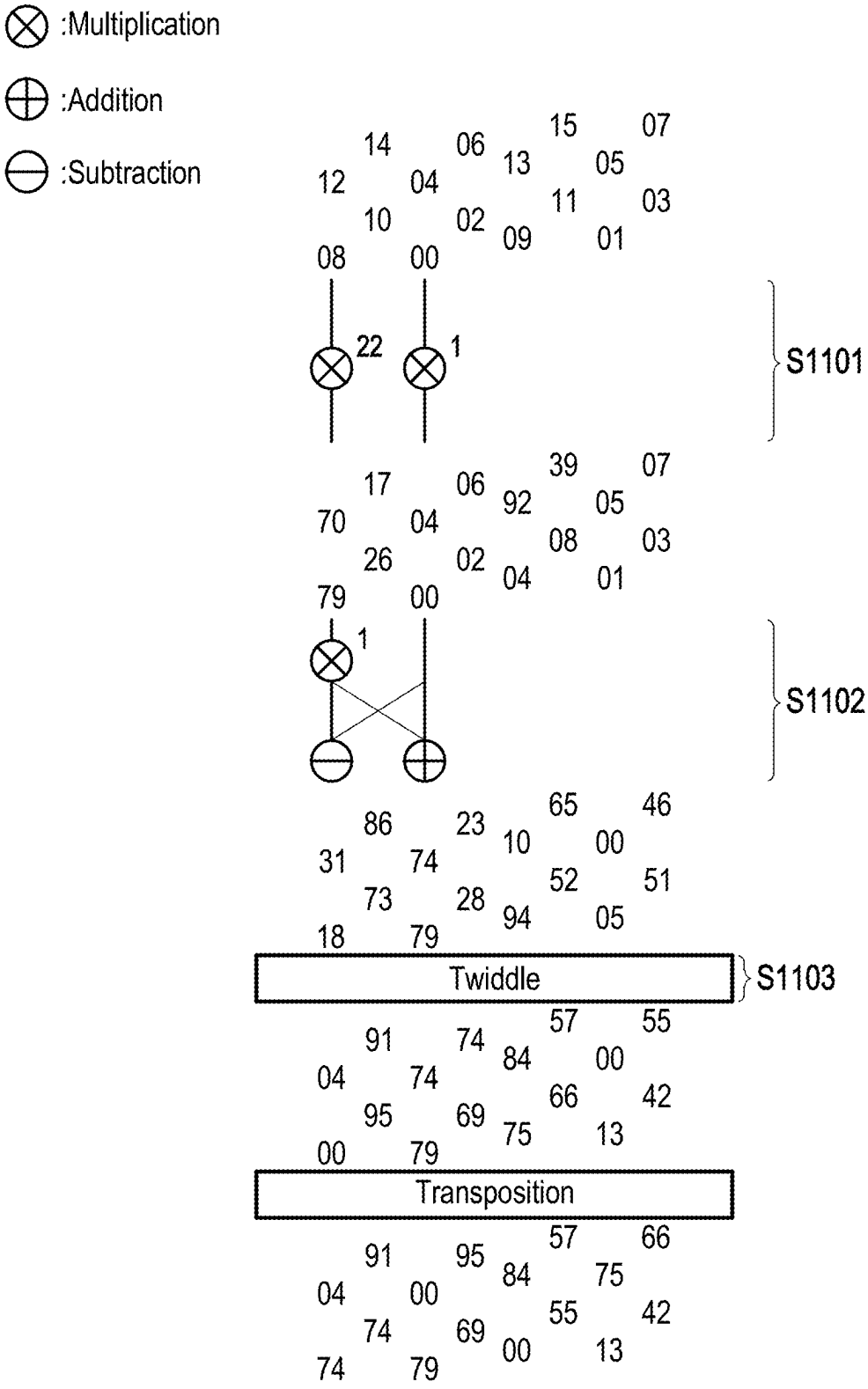

:Multiplication
:Addition
:Subtraction
14 06 15 07
12 04 13 05
10 02 11 03
08 00 09 01
22
1
S1101
17 06 39 07
70 04 92 05
26 02 08 03
79 00 04 01
1
S1102
86 23 65 46
31 74 10 00
73 28 52 51
18 79 94 05
Twiddle
S1103
91 74 57 55
04 74 84 00
95 69 66 42
00 79 75 13
Transposition
91 95 57 66
04 00 84 75
74 69 55 42
74 79 00 13

FIG. 11A

⊗ :Multiplication

⊕ :Addition

⊖ :Subtraction

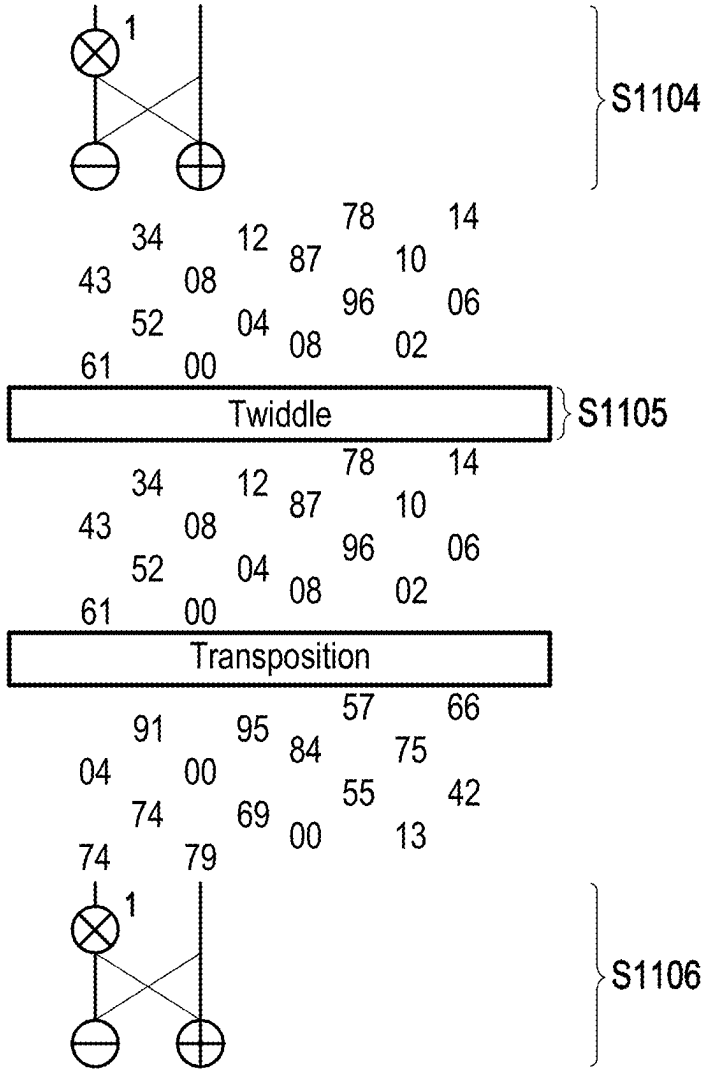


FIG. 11B

⊗ :Multiplication

⊕ :Addition

⊖ :Subtraction

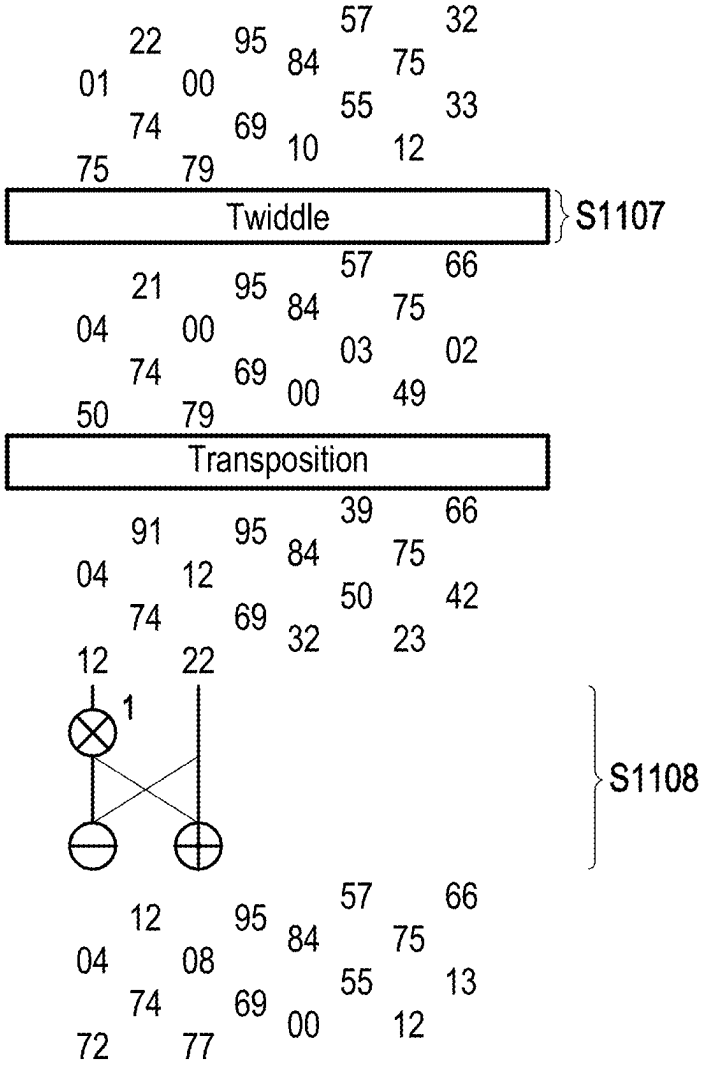


FIG. 11C

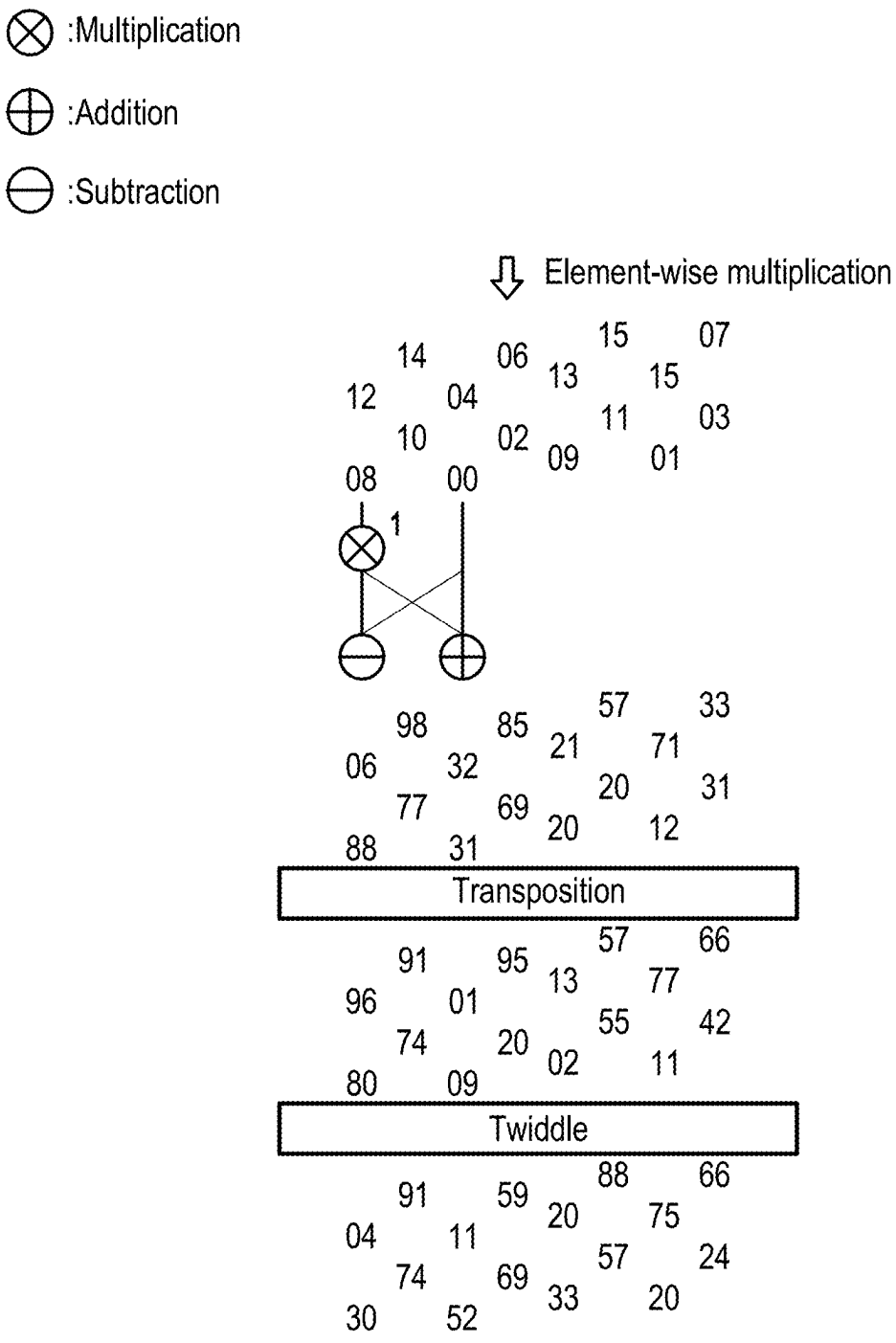

:Multiplication
:Addition
:Subtraction
Element-wise multiplication
14 06 15 07
12 04 13 15
10 02 11 03
08 00 09 01
1
98 85 57 33
06 32 21 71
77 69 20 31
88 31 20 12
Transposition
91 95 57 66
96 01 13 77
74 20 55 42
80 09 02 11
Twiddle
91 59 88 66
04 11 20 75
74 69 57 24
30 52 33 20

FIG. 11D

⊗ :Multiplication

⊕ :Addition

⊖ :Subtraction

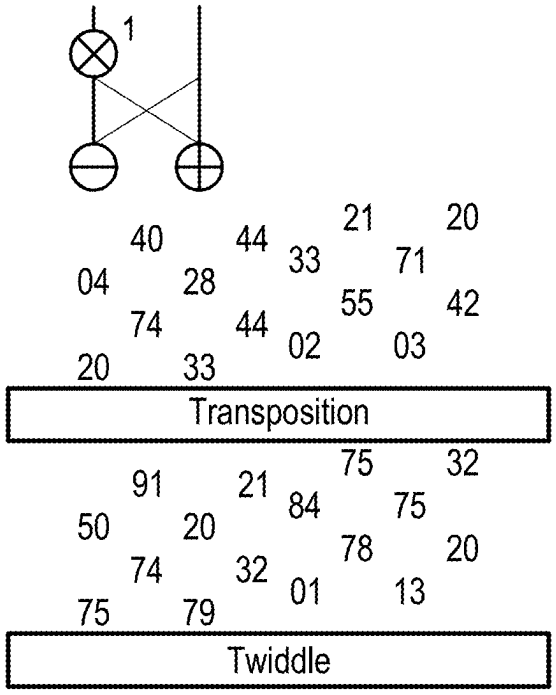


FIG. 11E

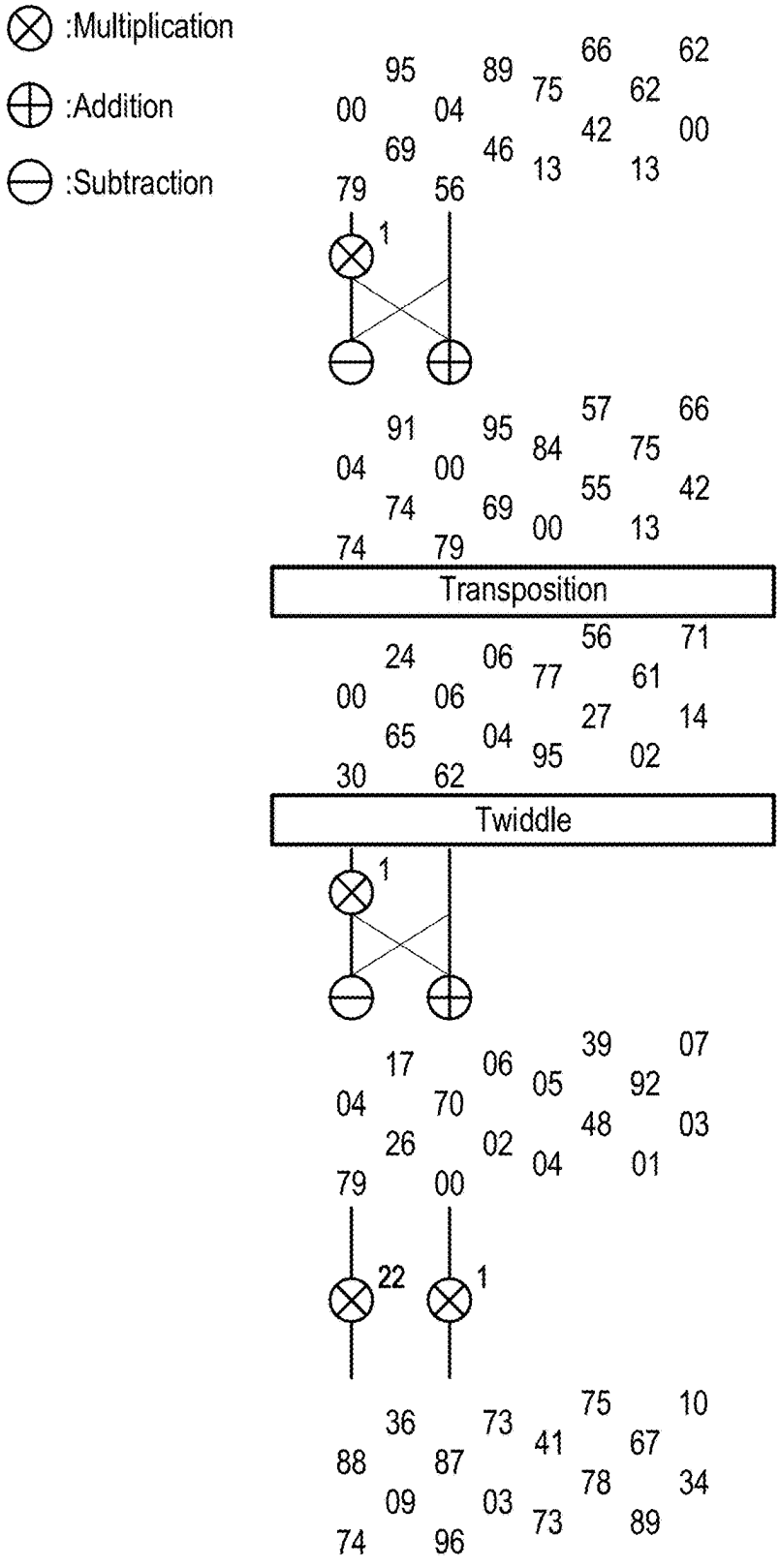

:Multiplication
:Addition
:Subtraction
95 89 66 62
00 04 75 62
69 46 42 00
79 56 13 13
1
91 95 57 66
04 00 84 75
74 69 55 42
74 79 00 13
Transposition
24 06 56 71
00 06 77 61
65 04 27 14
30 62 95 02
Twiddle
1
17 06 39 07
04 70 05 92
26 02 48 03
79 00 04 01
22
1
36 73 75 10
88 87 41 67
09 03 78 34
74 96 73 89

FIG. 11F

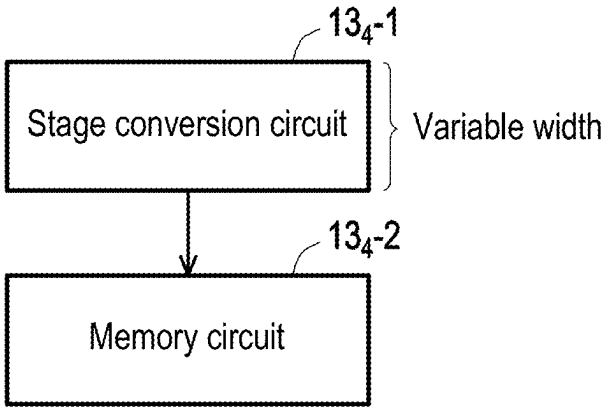

13_4-1
Stage conversion circuit
Variable width
13_4-2
Memory circuit

FIG. 12

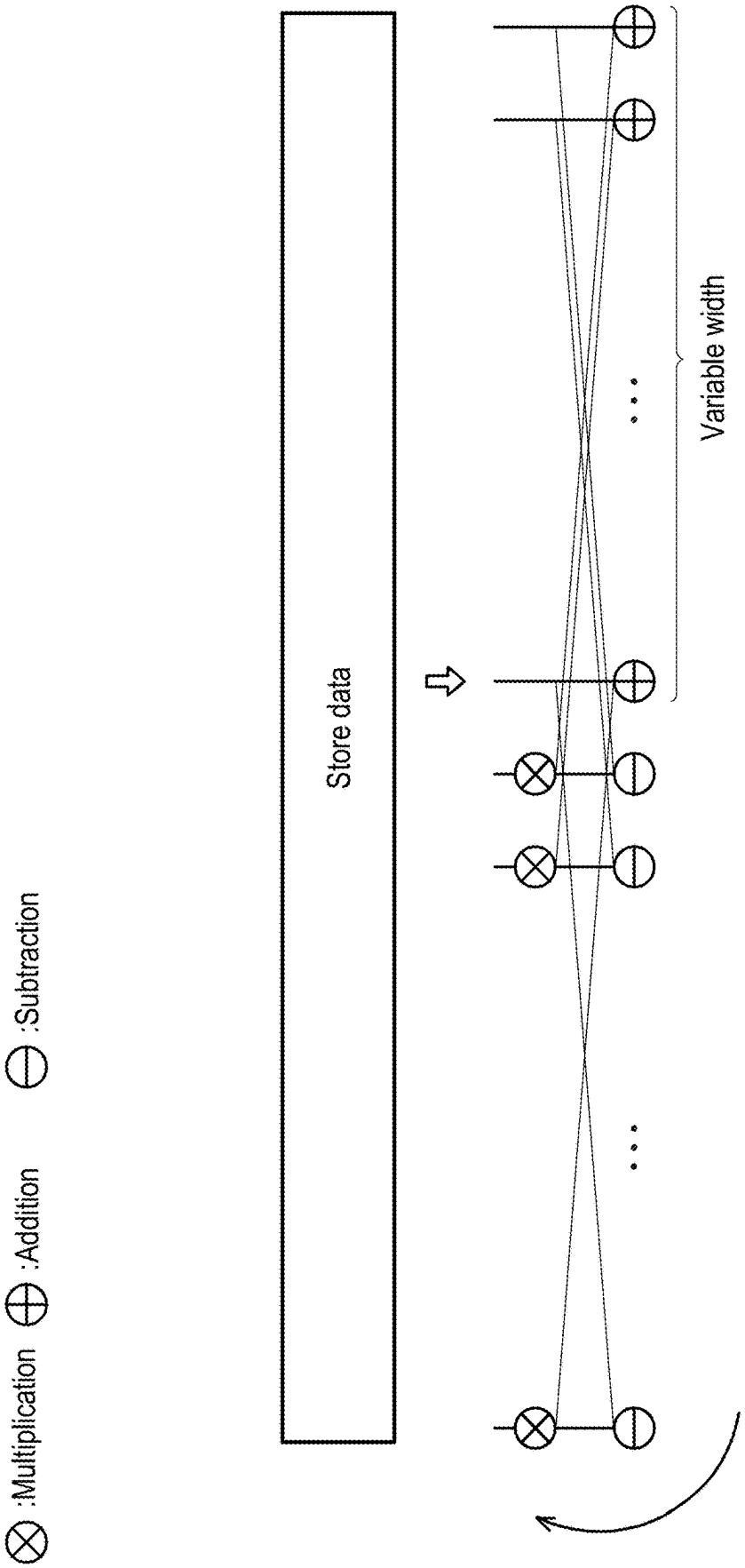

:Multiplication
:Addition
:Subtraction
Store data
Variable width

FIG. 13

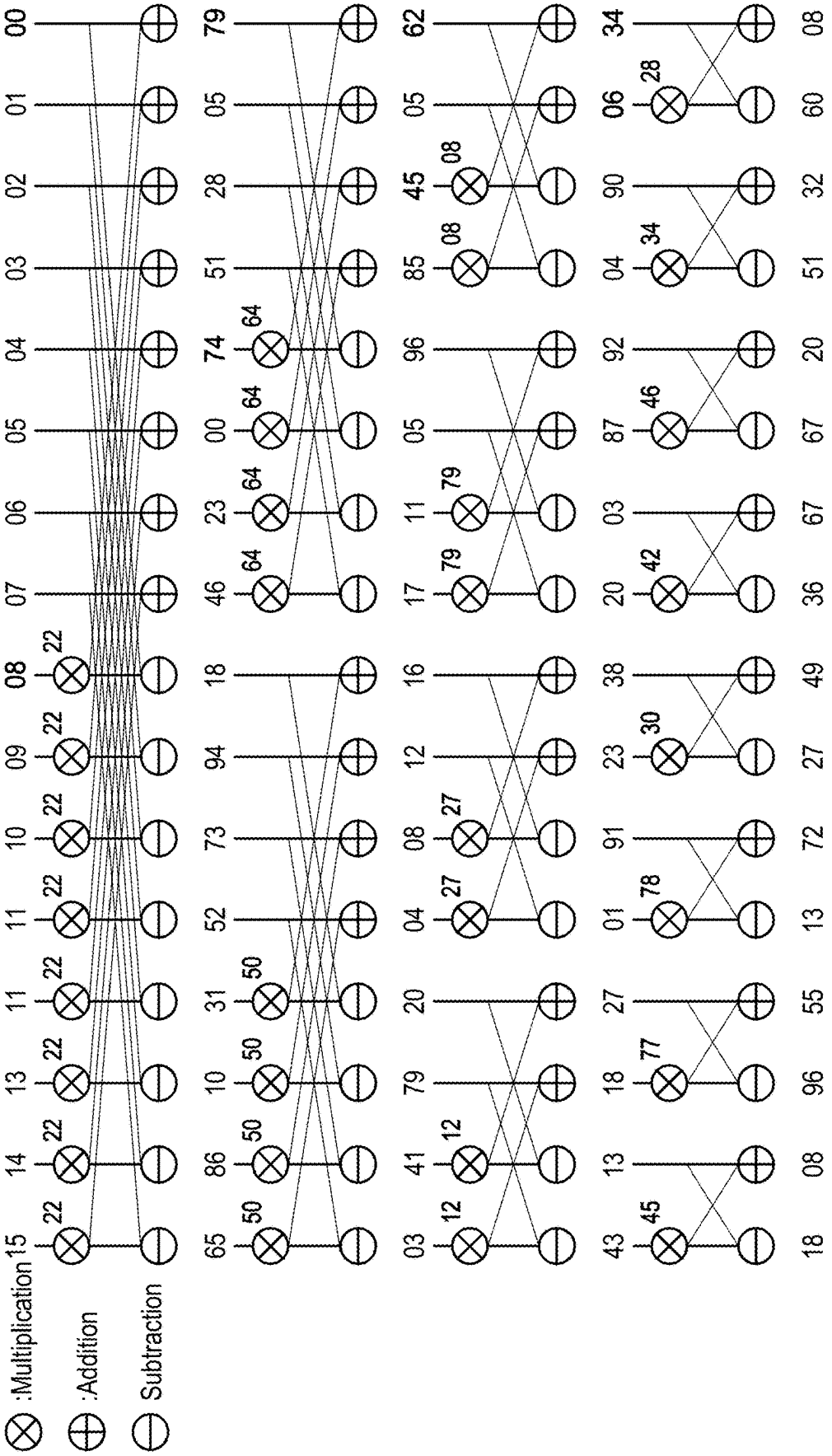


FIG. 14A

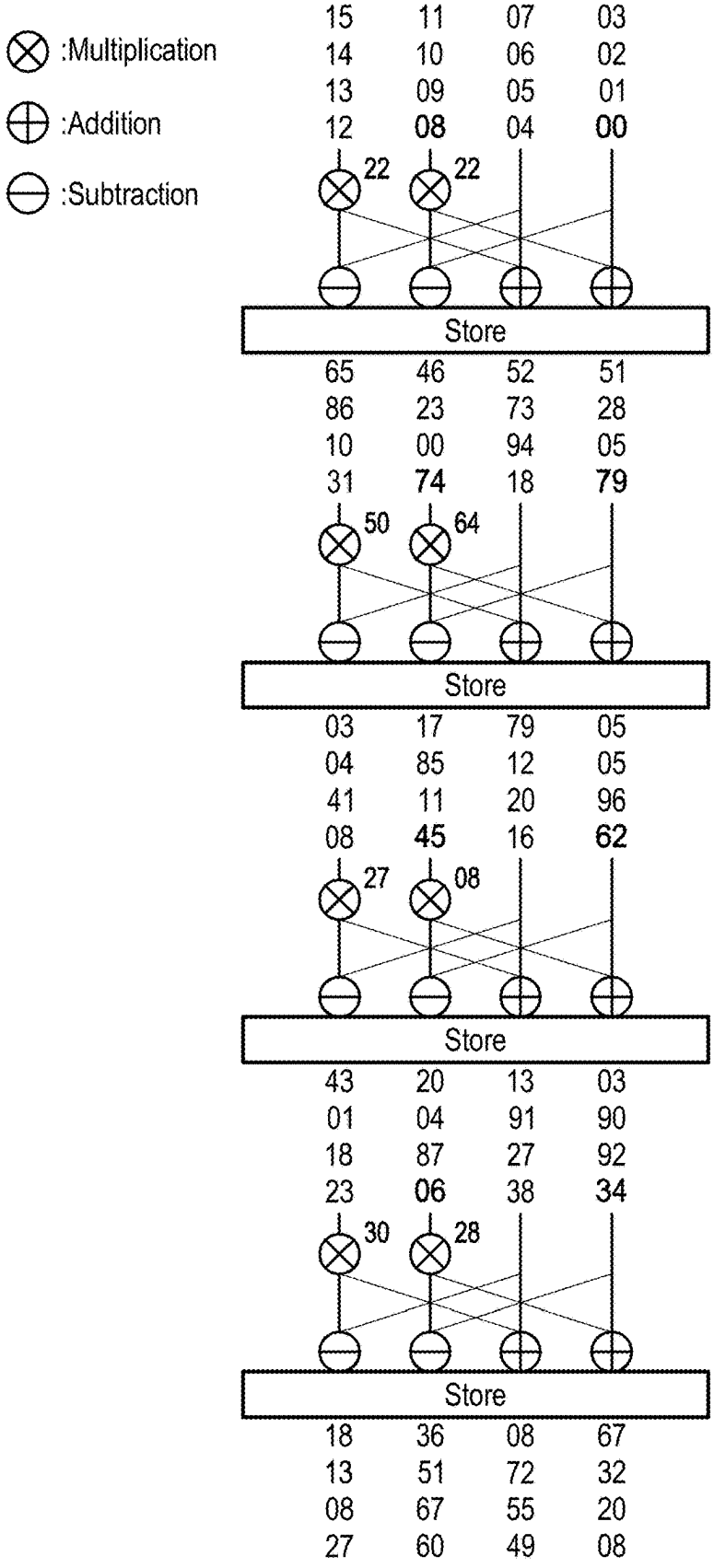

:Multiplication
:Addition
:Subtraction
15 11 07 03
14 10 06 02
13 09 05 01
12 08 04 00
22 22
Store
65 46 52 51
86 23 73 28
10 00 94 05
31 74 18 79
50 64
Store
03 17 79 05
04 85 12 05
41 11 20 96
08 45 16 62
27 08
Store
43 20 13 03
01 04 91 90
18 87 27 92
23 06 38 34
30 28
Store
18 36 08 67
13 51 72 32
08 67 55 20
27 60 49 08

FIG. 14B

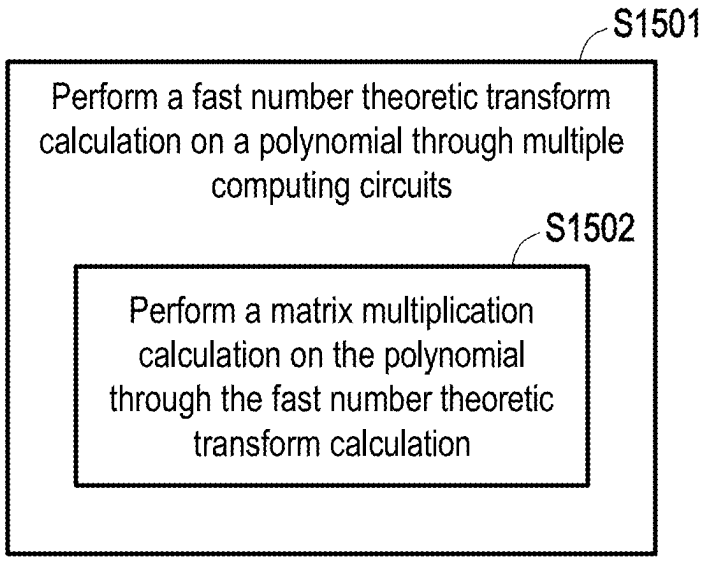

S1501
Perform a fast number theoretic transform calculation on a polynomial through multiple computing circuits
S1502
Perform a matrix multiplication calculation on the polynomial through the fast number theoretic transform calculation

FIG. 15

PROCESSING SYSTEM AND METHOD RELATED TO ENCRYPTION

CROSS-REFERENCE TO RELATED APPLICATION

[0001] This application claims the priority benefit of Taiwan application serial no. 113107218, filed on Feb. 29, 2024. The entirety of the above-mentioned patent application is hereby incorporated by reference herein and made a part of this specification.

BACKGROUND

Technical Field

[0002] The disclosure relates to a cryptographic technology, and in particular to a processing system and method related to encryption.

Description of Related Art

[0003] Today, data privacy and security issues have attracted much attention, especially when business data is stored in a hybrid multi-cloud environment and faces diverse security risks. However, there is an issue with traditional encryption methods, that is, data must be first decrypted before computation and analysis can be performed, which may lead to leakage of confidentiality during the process. In this case, the fully homomorphic encryption (FHE) technology has become a very promising solution.

[0004] Fully homomorphic encryption is a special encryption technology that allows calculations to be performed in an encrypted state and various operations to be executed on ciphertext without decrypting the ciphertext, which means that the calculations may be executed on encrypted data, and an obtained result is also encrypted. Only a person with a corresponding decryption key can decrypt the final result.

[0005] The main features of fully homomorphic encryption include:

[0006] Additive homomorphism: allow an addition operation to be performed on encrypted numbers, and an obtained result is still in an encrypted form without the need for decryption.

[0007] Multiplicative homomorphism: allow a multiplication computation to be performed on encrypted numbers, and an obtained result is still in an encrypted form also without the need for decryption.

[0008] Fully homomorphic: support both additive and multiplicative homomorphism, allowing an arbitrarily complex calculation on encrypted data without the need for decryption.

[0009] Solutions to achieve fully homomorphic encryption include but are not limited to Rivest-Shamir-Adleman (RSA) homomorphic encryption, Gentry's fully homomorphic encryption over the integers, Brakerski-Gentry-Vaikuntanathan (BGV) fully homomorphic encryption scheme, etc. It is worth noting that the RSA only has "multiplicative" homomorphism, but does not have "additive" homomorphism, so the RSA is a partially homomorphic encryption (PHE).

[0010] Although fully homomorphic encryption provides strong privacy protection and security, computing costs are usually high, resulting in relatively slow encryption and decryption processes while requiring a large amount of computing resources. Therefore, in practical applications, fully homomorphic encryption is still in the research and development stage and is being continuously improved to improve efficiency and effectiveness.

[0011] The existing fully homomorphic encryption technology still has many limitations in practical applications. For example, one of the major issues is computing efficiency. The traditional fully homomorphic encryption technology requires repeated decryption and encryption of data, introducing additional time and computing costs during the computing process. The inefficiency of such a computing manner becomes particularly obvious in scenarios such as real-time data processing, cloud computing, and the Internet of Things.

SUMMARY

[0012] The disclosure provides a processing system and method related to encryption, which can improve the computing efficiency of a homomorphic computation, encryption, and decryption.

[0013] A processing system related to encryption of an embodiment of the disclosure includes (but is not limited to) multiple computing circuits. A memory stores data. The computing circuit performs a fast number theoretic transform (NTT) calculation on a polynomial. A matrix multiplication calculation is performed on the polynomial through the fast number theoretic transform calculation.

[0014] A method related to encryption of an embodiment of the disclosure includes (but is not limited to) the following steps. A fast number theoretic transform calculation is performed on a polynomial through multiple computing circuits. A matrix multiplication calculation is performed on the polynomial through the fast number theoretic transform calculation.

[0015] In order for the features and advantages of the disclosure to be more comprehensible, the following specific embodiments are described in detail in conjunction with the drawings.

BRIEF DESCRIPTION OF THE DRAWINGS

[0016] FIG. **1** is an element block diagram of a processing system according to an embodiment of the disclosure.

[0017] FIG. **2** is a schematic diagram of a butterfly computation according to an embodiment of the disclosure.

[0018] FIG. **3** is a system architecture diagram of two-dimensional fast number theoretic transform according to an embodiment of the disclosure.

[0019] FIG. **4**A is a schematic diagram illustrating a butterfly computation according to an embodiment of the disclosure.

[0020] FIG. **4**B is a schematic diagram illustrating a multiplication computation with a twiddle factor according to an embodiment of the disclosure.

[0021] FIG. **4**C is a schematic diagram illustrating factor position swapping according to an embodiment of the disclosure.

[0022] FIG. **5**A to FIG. **5**C are schematic diagrams illustrating one-dimensional fast number theoretic transform according to an embodiment of the disclosure.

[0023] FIG. **6**A to FIG. **6**D are schematic diagrams illustrating two-dimensional fast number theoretic transform according to an embodiment of the disclosure.

[0024] FIG. **7** is a system architecture diagram of three-dimensional fast number theoretic transform according to an embodiment of the disclosure.

[0025] FIG. **8**A is a schematic diagram illustrating a multiplication computation with a twiddle factor according to an embodiment of the disclosure.

[0026] FIG. **8**B is a schematic diagram illustrating memory address change according to an embodiment of the disclosure.

[0027] FIG. **9**A to FIG. **9**F are schematic diagrams illustrating three-dimensional fast number theoretic transform according to an embodiment of the disclosure.

[0028] FIG. **10** is a system architecture diagram of four-dimensional fast number theoretic transform according to an embodiment of the disclosure.

[0029] FIG. **11**A to FIG. **11**F are schematic diagrams illustrating four-dimensional fast number theoretic transform according to an embodiment of the disclosure.

[0030] FIG. **12** is a system architecture diagram of non-pipelined fast number theoretic transform according to an embodiment of the disclosure.

[0031] FIG. **13** is a schematic diagram illustrating data storage according to an embodiment of the disclosure.

[0032] FIG. **14**A is a schematic diagram illustrating forward fast number theoretic transform according to an embodiment of the disclosure.

[0033] FIG. **14**B is a schematic diagram illustrating non-pipelined fast number theoretic transform according to an embodiment of the disclosure.

[0034] FIG. **15** is a flowchart of an encryption method according to an embodiment of the disclosure.

DESCRIPTION OF THE EMBODIMENTS

[0035] FIG. **1** is a block diagram of a processing system **10** according to an embodiment of the disclosure. Please refer to FIG. **1**. The processing system **10** includes (but is not limited to) one or more memories **11**, a data conversion circuits **12**, and multiple computing circuits **13**.

[0036] The processing system **10** may be an electronic device such as a server, a desktop computer, a notebook computer, a smart phone, a tablet computer, a wearable device, a vehicle-mounted device, or a smart home appliance, or may be a central processing unit (CPU), a graphics processing unit (GPU), a programmable general-purpose or specific-purpose microprocessor, a digital signal processor (DSP), a programmable controller, a field programmable gate array (FPGA), an application-specific integrated circuit (ASIC), a computing accelerator, other similar elements, or a combination of the above elements.

[0037] The memory **11** may be any type of fixed or removable random access memory (RAM), read only memory (ROM), or similar elements. In an embodiment, the memory **11** is used to store program codes, software modules, configurations, data (for example, factors, algorithms, matrices, or parameters), or files, and an embodiment thereof will be described in detail later.

[0038] The data conversion circuit **12** is coupled to the memory **11**. The data conversion circuit **12** may be various types of processors, controllers, processing elements (PE), processing cores, FPGA, ASIC, or other digital circuits.

[0039] The computing circuit **13** is coupled to the memory **11** and the data conversion circuit **12**. The computing circuit **13** may be various types of processors, controllers, processing elements, processing cores, FPGA, ASIC, or other digital circuits.

[0040] In the following, the embodiments of the disclosure will be further explained below in conjunction with various devices, elements, and modules in the processing system **10**. Each process of the embodiments of the disclosure may be adjusted according to the implementation situation.

[0041] The main concept of fully homomorphic encryption is to perform a computation on ciphertext to obtain an encrypted result, and a result obtained by decrypting the encrypted result is consistent with a result of performing the same computation on plaintext. It is worth noting that the basic computation of fully homomorphic encryption usually involves a multiplication computation of a polynomial. Fast number theoretic transform (NTT) is an algorithm that implements polynomial multiplication. A matrix multiplication calculation may be performed on a polynomial through a fast number theoretic transform calculation. Fast number theoretic transform is the fast Fourier transform (FFT) in integer form. Fast number theoretic transform may reduce the original time complexity of degree N polynomial multiplication from O(N^2) to O(NlogN). For example, fast number theoretic transform converts a polynomial computation into a point computation in a fast number theoretic transform domain, thereby greatly improving computing efficiency.

[0042] The mathematical expression of fast number theoretic transform may be:

$$X_j = \sum_i (\omega^j)^i x_i \tag{1}$$

[0043] where x_i is an i-th data vector, ω^j is a factor of a j-th degree N polynomial (may be presented by a matrix that records factors in the polynomial), and X_j is a j-th output matrix. Equation (1) above is matrix-vector multiplication, and the matrix used may be a Vandermonde matrix.

[0044] Since ω is periodic, Equation (1) above may be further simplified to only require multiplication of O(NlogN) to complete all computations. For example, FIG. **2** is a schematic diagram of a butterfly computation according to an embodiment of the disclosure. Please refer to FIG. **2**. Fast number theoretic transform/fast Fourier transform may be implemented through a butterfly architecture. In the multiplication of two polynomials, $2x^3+7x^2+1x^1+8x^0$ and $2x^3+0x^2+4x^1+8x^0$, the polynomial $2x^3+7x^2+1x^1+8x^0$ is converted into a data vector [2, 7, 1, 8], and the polynomial $2x^3+0x^2+4x^1+8x^0$ is converted into a data vector [2, 0, 4, 8]. The two data vectors are first converted into the fast number theoretic transform domain via time complexity of O(NlogN), and a point multiplication computation (that is, element-wise multiplication) is performed in the fast number theoretic transform domain. The point multiplication computation only takes time complexity of O(N). Finally, time complexity of O(NlogN) is taken to convert back to the normal domain. It should be noted that in the drawing, "x" is multiplication (multiply by the factor on the side, such as 2×4), "+" is addition (add two values on a connecting line, such as (7×4)+8), and "−" is subtraction (subtract two values on a connecting line, such as (2×4)−1).

[0045] In addition, in the drawing, the polynomial x^N+1 is taken as an example, so FIG. **2** shows negacyclic fast number theoretic transform. The polynomial x^N+1 is substituted into Equation (1) to become $X_j=\Sigma_j(\omega^{2j+1})^i x_i$, so ω becomes a degree 2N polynomial and may be implemented using different conversion manners.

[0046] In an embodiment, in the computation of fast number theoretic transform, there are log N stages (NlogN multiplications), and the hardware implementation includes full pipelined and non-full pipelined manners.

[0047] The full pipelined manner allocates the hardware (for example, the computing circuit **13**, the memory **11**, or a combination thereof) required by all stages, so more hardware resources are available for use, and there is better performance in output/cycle.

[0048] The non-pipelined manner may use the same set of hardware to implement different stages of operations, so there are fewer hardware resources available for use, but control is easier.

[0049] In an embodiment, the computing circuits **13** perform a fast number theoretic transform calculation on a polynomial. A matrix multiplication calculation is performed on the polynomial through the fast number theoretic transform calculation.

[0050] In an embodiment, the computing circuits **13** perform multi-dimensional fast number theoretic transform. The multi-dimensional fast number theoretic transform converts matrix-vector multiplication into a computation architecture of a multi-stage matrix multiplication calculation through multiple sub-computations. One computing circuit **13** performs one sub-computation, and each sub-computation is located at one stage of the multi-stage computation architecture.

[0051] The mathematical expression of multi-dimensional fast number theoretic transform is:

$$X_{k_1,k_2,\ldots,k_d} = \sum_{n_1=0}^{N_1-1}\left(\omega_{N_1}^{k_1 n_1}\sum_{n_2=0}^{N_2-1}\omega_{N_2}^{k_2 n_2}\cdots\sum_{n_d=0}^{N_d-1}\omega_{N_d}^{k_d n_d}\cdot x_{n_1,n_2,\ldots,n_d}\right) \quad (2)$$

[0052] where $X_{k_1,k_2,\ldots,k_d}$ is the output factor of $(k_1, k_2, \ldots, k_d)$-th multi-dimensional fast number theoretic transform, $x_{n_1,n_2,\ldots,n_d}$ is the $(n_1, n_2, \ldots, n_d)$-th input matrix, ω is a twiddle factor, $N_1*N_2*\ldots*N_d$ is N, dis a target dimension, and N is the degree of the polynomial.

[0053] FIG. **3** is a system architecture diagram of two-dimensional fast number theoretic transform according to an embodiment of the disclosure. Please refer to FIG. **3**, which shows a two-dimensional pipelined multi-dimensional fast number theoretic transform architecture. For a two-dimensional computation (d of Equation (2) is 2), the mathematical expression of multi-dimensional fast number theoretic transform is:

$$X_{N_2k_1+k_2} = \sum_{n_1=0}^{N_1-1}\sum_{n_2=0}^{N_2-1}x_{N_1n_2+n_1}\left(\omega^{2(N_2k_1+k_2)+1}\right)^{N_1n_2+n_1} = \sum_{n_1=0}^{N_1-1}\left(\left(\sum_{n_2=0}^{N_2-1}x_{N_1n_2+n_1}\left(\omega^{2k_2+1}\right)^{N_1n_2}\right)\omega^{n_1(2k_2+1-N_2)}\right)\left(\omega^{2k_1+1}\right)^{N_2n_1} \quad (3)$$

[0054] where $X_{N_2k_1+k_2}$ is the output factor of $(N_2k_1+k_2)$-th multi-dimensional fast number theoretic transform, $x_{N_1n_2+n_1}$ is the $(N_1n_2+n_1)$-th input matrix, ω is the twiddle factor, N_1*N_2 is N, and N is the degree of the polynomial.

[0055] Equation (3) may be allocated to the computing circuits **13** for implementation. The computing circuits **13** include one or more stage conversion circuits, one or more first twiddle circuits, and one or more transposition circuits. Taking FIG. **3** as an example, the computing circuits **13** with such an architecture include two stage conversion circuits **13$_1$-1** and **13$_1$-4**, a first twiddle circuit **13$_1$-2**, and a transposition circuit **13$_1$-3**. The order of execution of the computing circuits **13** is the stage conversion circuit **13$_1$-1**, the first twiddle circuit **13$_1$-2**, the transposition circuit **13$_1$-3**, and the stage conversion circuit **13$_1$-4**.

[0056] In an embodiment, the sub-computations of the stage conversion circuits **13$_1$-1** and **13$_1$-4** are butterfly computations. FIG. **4**A is a schematic diagram illustrating a butterfly computation according to an embodiment of the disclosure. Please refer to FIG. **4**A. A sub-stage of the butterfly computation is related to the degree (N) of the polynomial, that is, log N. In an m-th sub-stage, the butterfly computation of

$$\sqrt[d]{N}/(2\hat{}m)$$

points is performed, where d is the target dimension (two-dimensional as shown in FIG. **3**, that is, d is 2). For example, in a first sub-stage, one of the outputs is $x_{1-1-00}+x_{1-1-(i1+1)}\cdot\omega 0$(each multiplication "×" has a corresponding twiddle factor ω), where i1+1 is M/2+0. A data vector $[x_{1-1-00}, \sim, x_{1-1-M}]$ has M values, and M is $\sqrt[d]{N}$. By analogy, another output is $x_{1-1-01}+x_{1-1-(i1+2)}\cdot\omega 1$. The next output is $x_{1-1-i1}+x_{1-1-M}\cdot\omega 2$. Finally, the output factors of a log N-th sub-stage are X_{1-2-00}, X_{1-2-01}, X_{1-2-02}, X_{1-2-03}, . . . , $X_{1-2-(M-1)}$, X_{1-2-M}.

[0057] In an embodiment, the sub-computation of the first twiddle circuit **13$_1$-2** is a multiplication computation of the twiddle factor. FIG. **4**B is a schematic diagram illustrating a multiplication computation with a twiddle factor according to an embodiment of the disclosure. Please refer to FIG. **4**B. A multiplication factor is multiplied by the twiddle factor, and a multiplication result is used as a starting twiddle factor. For example, the starting twiddle factor is ωˆ(n(1−N_2)), and the multiplication factor is nˆ(2n), where n is 0~N1−1}. In addition, the multiplication result is further multiplied by the input factor.

[0058] In an embodiment, the sub-computation of the transposition circuit **13$_1$-3** is factor position swapping. FIG. **4**C is a schematic diagram illustrating factor position swapping according to an embodiment of the disclosure. Please refer to FIG. **4**C. The sub-computation performs transposition on a matrix

$$\begin{bmatrix} A & B \\ C & D \end{bmatrix},$$

that is,

$$\begin{bmatrix} A & B \\ C & D \end{bmatrix}^T = \begin{bmatrix} A^T & C^T \\ B^T & D^T \end{bmatrix}.$$

First, the positions of the matrices A and C are swapped, so that the matrix C is stored in a temporary storage area/register of the memory **11**, and the matrices A and D are stored in another temporary storage area/register of the memory **11**. If the matrix C is stored, storing of the matrix B will be skipped. If the matrices A and B are input at the same time, the positions of the matrices A and B will be swapped. Then, positions are swapped again. For example, if the matrices A and B are not input, the positions will not be swapped.

[0059] Please refer to FIG. **3**. The data conversion circuit **12** obtains the input matrix. The column number or the row number of the input matrix is $\sqrt[d]{N}$, such as $\sqrt{N}$. Multiple elements of the input matrix are factors of the polynomial used for encryption, such as the factors of the polynomial related to ciphertext or plaintext used for fully homomorphic encryption. In an embodiment, the elements of the input matrix are respectively the same as multiple elements in input data (one-to-one element correspondence). The input data is also the factors of the same polynomial and is in vector form. The input data is, for example, data related to fully homomorphic encrypted ciphertext or plaintext.

[0060] In an embodiment, the data conversion circuit **12** converts the input data into the input matrix according to the target dimension. The target dimension is greater than one dimension (two dimensions in the embodiment). Therefore, the data conversion circuit **12** vectorizes the factors of the polynomial into the input data, and converts the input data into the two-dimensional input matrix. The elements in the input matrix are the elements in the input data in vector form (values thereof are the factors of the polynomial). In other words, "converting" a vector into a matrix is to allocate the elements in the input data to corresponding positions in the input matrix. For example, a first element of the input data is allocated to a first row and a first column of the input matrix.

[0061] In response to the target dimension being two-dimensional, the mathematical expression of multi-dimensional fast number theoretic transform is determined as Equation (3). Next, the butterfly computation is performed on the input matrix through the stage conversion circuit $\mathbf{13_1}$**-1** of log $\sqrt{N}$ sub-stages, the multiplication computation of the twiddle factor is performed on the output factor of the butterfly computation through the first twiddle circuit $\mathbf{13_1}$**-2**, factor position swapping is performed on the output factor of the multiplication computation through the transposition circuit $\mathbf{13_1}$**-3**, and the butterfly computation is performed on the output factor of position transposition through the stage conversion circuit $\mathbf{13_1}$**-4** of log $\sqrt{N}$ sub-stages. That is, two-dimensional multi-dimensional fast number theoretic transform is performed on the input matrix through the computing circuits **13** (that is, the two stage conversion circuits $\mathbf{13_1}$**-1** and $\mathbf{13_1}$**-4**, the first twiddle circuit $\mathbf{13_1}$**-2**, and the transposition circuit $\mathbf{13_1}$**-3**).

[0062] FIG. **5**A to FIG. **5**C are schematic diagrams illustrating one-dimensional fast number theoretic transform according to an embodiment of the disclosure. Please refer to FIG. **5**A to FIG. **5**C. For example, the input data is [00, 01, 02, 03, 14, 05, 06, 07, 08, 09, 10, 11, 12]. It should be noted that in the drawings, "×" is multiplication (multiply by the twiddle factor on the side, such as 15×22), "+" is addition (add two values on a connecting line, such as (15×22)+08), and "−" is subtraction (subtract two values on a connecting line, such as (15×22)−08). The twiddle factor of a first stage is (−1)^(½) (for example, 22(mod 97)), and the twiddle factors of a second stage are (−1)^(¼) (for example, 64 (mod 97)) and (−1)^(¾) (for example, 50(mod 97)), and so on. Finally, the output of the input data after fast number theoretic transform is [96, 89, 34, 87, 67, 73, 10, 74, 73, 09, 78, 88, 41, 36, 75].

[0063] FIG. **6**A to FIG. **6**D are schematic diagrams illustrating two-dimensional fast number theoretic transform according to an embodiment of the disclosure. Please refer to FIG. **6**A to FIG. **6**D. The degree of the polynomial is $N=N_1N_2$(for example, $N_1=N_2=\sqrt{N}$). An index of the polynomial is represented by $x_{N_1n_2+n_1}$. An inner layer n_2 is added up to fast number theoretic transform of the first stage. $\omega^{n_1(2k_2+1-N_2)}$ is the twiddle factor used in the multiplication computation. An outer layer n_1 is added up to fast number theoretic transform of the second stage. The hardware architecture of FIG. **3** is suitable for implementing both forward and inverse fast number theoretic transform, and the difference is that the remaining hardware pipelined part may be shared by adding one or two additional stages of specific multiplication computations.

[0064] The data conversion circuit converts 1×16 input data into a 4×4 input matrix, such as

$$\begin{bmatrix} 15 & 11 & 07 & 03 \\ 14 & 10 & 06 & 02 \\ 13 & 09 & 05 & 01 \\ 12 & 08 & 04 & 00 \end{bmatrix}.$$

[0065] In Step S**601**, the multiplication computation is performed on the twiddle factor $(\omega^{2k_2+1})^{N_1n_2}$. In Step S**602**, the multiplication computation is performed on the twiddle factor $\omega^{n_1(2k_2+1-N_2)}$. In Step S**603**, the multiplication computation is performed on the twiddle factor $(\omega^{2k_2+1})^{N_1n_2}$. The "twiddle" processing in the drawings is, for example, the sub-computation shown in FIG. **4**B, that is, the multiplication computation is performed on the corresponding twiddle factor. In addition, the "transposition" processing in the drawings is, for example, the sub-computation shown in FIG. **4**C, that is, factor position swapping. Finally, the output matrix is

$$\begin{bmatrix} 75 & 78 & 10 & 34 \\ 36 & 09 & 73 & 03 \\ 41 & 73 & 67 & 89 \\ 88 & 74 & 87 & 96 \end{bmatrix},$$

and the values are the same as the values in the output vector of FIG. **5**C (the difference is only in the positions of the matrix).

[0066] The twiddle factors in Equation (3) are ω^{2k+1} {k=0~N_2−1} and $\omega^{n_1(2k_2+1-N_2)}$. In an embodiment, the twiddle factors may be prestored in the memory **11** for used in subsequent computations. In another embodiment, some or all of the twiddle factors may be generated on the fly or dynamically. For example, the twiddle factors $\omega^{n(1-N_2)}$ and

$\omega^{2n}\{n=0\sim N_1-1\}$ may be computed on the fly to generate the twiddle factor $\omega^{n_1(2k_2+1-N_2)}$. At this time, the amount of data required to be stored is $3\sqrt[3]{N}$.

[0067] FIG. **7** is a system architecture diagram of three-dimensional fast number theoretic transform according to an embodiment of the disclosure. Please refer to FIG. **7**, which shows a three-dimensional pipelined multi-dimensional fast number theoretic transform architecture. For a three-dimensional computation (d of Equation (2) is 3), the mathematical expression of multi-dimensional fast number theoretic transform is:

$$X_{N_3N_2k_1+N_3k_2+k_3} = \sum_{n_1=0}^{N_1-1}\sum_{n_2=0}^{N_2-1}\sum_{n_3=0}^{N_3-1} x_{N_1N_2n_3+N_1n_2+n_1} \left(\omega^{2(N_3N_2k_1+N_3k_2+k_3)+1}\right)^{N_1N_2n_3+N_1n_2+n_1} = \sum_{n_1=0}^{N_1-1}\left(\sum_{n_2=0}^{N_2-1}\left(\left(\sum_{n_3=0}^{N_3-1} x_{N_1N_2n_3+N_1n_2+n_1}\,\omega^{N_1N_2n_3}\left(\omega^{2k_3}\right)^{N_1N_2n_3}\right)\left(\omega^{2k_3+1}\right)^{N_1n_2+n_1}\left(\omega^{2k_2}\right)^{N_3N_1n_2}\right)\right)\left(\omega^{2N_3k_2}\right)^{N_1n_2+n_1}\left(\omega^{2k_1}\right)^{N_3N_2n_1} \quad (4)$$

[0068] where $X_{N_3N_2k_1+N_3k_2+k_3}$ is the output factor of $(N_3N_2k_1+N_3k_2+k_3)$-th multi-dimensional fast number theoretic transform, $x_{N_1N_2n_3+N_1n_2+n_1}$ is the $(N_1N_2n_3+N_1n_2+n_1)$-th input matrix, ω is the twiddle factor, $N_1*N_2*N_3$ is N and N is the degree of the polynomial.

[0069] Equation (4) may be allocated to the computing circuits **13** for implementation. The computing circuits **13** include one or more first twiddle circuits, one or more second twiddle circuits, one or more stage conversion circuits, one or more transposition circuits, and one or more memory transposition circuits. Taking FIG. **7** as an example, the computing circuits **13** with such an architecture include two first twiddle circuits $\mathbf{13}_2$**-1** and $\mathbf{13}_2$**-13**, two second twiddle circuits $\mathbf{13}_2$**-2** and $\mathbf{13}_2$**-12**, three stage conversion circuits $\mathbf{13}_2$**-3**, $\mathbf{13}_2$**-7**, and $\mathbf{13}_2$**-11**, four transposition circuits $\mathbf{13}_2$**-4**, $\mathbf{13}_2$**-6**, $\mathbf{13}_2$**-8**, and $\mathbf{13}_2$**-10**, and two memory transposition circuits $\mathbf{13}_2$**-5** and $\mathbf{13}_2$**-9**. The order of execution of the computing circuits **13** is the first twiddle circuit $\mathbf{13}_2$**-1**, the second twiddle circuit $\mathbf{13}_2$**-2**, the stage conversion circuit $\mathbf{13}_2$**-3**, the transposition circuit $\mathbf{13}_2$**-4**, the memory transposition circuit $\mathbf{13}_2$**-5**, the transposition circuit $\mathbf{13}_2$**-6**, the stage conversion circuit $\mathbf{13}_2$**-7**, the transposition circuit $\mathbf{13}_2$**-8**, the memory transposition circuit $\mathbf{13}_2$**-9**, the transposition circuit $\mathbf{13}_2$**-10**, the stage conversion circuit $\mathbf{13}_2$**-11**, the second twiddle circuit $\mathbf{13}_2$**-12**, and the first twiddle circuit $\mathbf{13}_2$**-13**.

[0070] In an embodiment, the sub-computations of the first twiddle circuits $\mathbf{13}_2$**-1** and $\mathbf{13}_2$**-13** are the multiplication computations of the twiddle factors, and reference may be made to the description of FIG. **4**B.

[0071] In an embodiment, the sub-computations of the second twiddle circuits $\mathbf{13}_2$**-2** and $\mathbf{13}_2$**-12** are the multiplication computation of the twiddle factors. FIG. **8**A is a schematic diagram illustrating a multiplication computation with a twiddle factor according to an embodiment of the disclosure. Please refer to FIG. **8**A. The output of the previous stage is used as the input, and the input is multiplied by the twiddle factor to obtain the output.

[0072] In an embodiment, the sub-computations of the stage conversion circuits $\mathbf{13}_2$**-3**, $\mathbf{13}_2$**-7**, and $\mathbf{13}_2$**-11** are the butterfly computations, and reference may be made to the description of FIG. **4**A.

[0073] In an embodiment, the sub-computations of the transposition circuits $\mathbf{13}_2$**-4**, $\mathbf{13}_2$**-6**, $\mathbf{13}_2$**-8**, and $\mathbf{13}_2$**-10** are factor position swapping, and reference may be made to the description of FIG. **4**C.

[0074] In an embodiment, the sub-computations of the memory transposition circuits $\mathbf{13}_2$**-5** and $\mathbf{13}_2$**-9** are memory address change. FIG. **8**B is a schematic diagram illustrating memory address change according to an embodiment of the disclosure. Please refer to FIG. **8**B. Two 4×4 matrices (for example

$$\begin{bmatrix} 31 & 30 & 29 & 28 \\ 27 & 26 & 25 & 24 \\ 23 & 22 & 21 & 20 \\ 19 & 18 & 17 & 16 \end{bmatrix} \text{ and } \begin{bmatrix} 15 & 14 & 13 & 12 \\ 11 & 10 & 09 & 08 \\ 07 & 06 & 05 & 04 \\ 03 & 02 & 01 & 00 \end{bmatrix})$$

on the left side of the drawing are extended and combined into a 8×4 matrix on the right side of the drawing. Next, a write address of each element in the 8×4 matrix in the memory **11** is changed. For example, each column in the matrix is converted to binary form. Taking the 8×4 matrix as an example, elements in each column are converted into three bits: abc are converted into bca (001->010).

[0075] Please refer to FIG. **7**. The data conversion circuit **12** converts the input data into the input matrix according to the target dimension. The input data is the factors of the polynomial and is in vector form. The input data is, for example, data related to fully homomorphic encrypted ciphertext or plaintext. In addition, the target dimension is greater than one dimension (three dimensions in the embodiment). Therefore, the data conversion circuit **12** vectorizes the factors of the polynomial into the input data, and converts the input data into the two-dimensional input matrix. The column number or the row number of the input matrix is

$$\sqrt[d]{N},$$

such as

$$\sqrt[3]{N}.$$

[0076] In response to the target dimension being three-dimensional, the mathematical expression of multi-dimensional fast number theoretic transform is determined as Equation (4). Then, the multiplication computation is performed on the input matrix and the corresponding twiddle factor through the first twiddle circuit $\mathbf{13}_2$**-1**, the multiplication computation is performed on the output factor of the multiplication computation and the corresponding twiddle factor through the second twiddle circuit $\mathbf{13}_2$**-2**, the butterfly computation is performed on the output factor of the multiplication computation through the stage conversion circuit $\mathbf{13}_2$**-3** of $\log \sqrt[3]{N}$ sub-stages, factor position swapping is performed on the output factor of the butterfly computation through the transposition circuit $\mathbf{13}_2$**-4**, memory address change is performed on the position-swapped factor through the memory transposition circuit $\mathbf{13}_2$**-5**, factor position swapping is performed on the address-changed factor

through the transposition circuit **13$_2$-6**, the butterfly computation is performed on the position-swapped factor through the stage conversion circuit **13$_2$-7** of log $\sqrt[3]{N}$ sub-stages, factor position swapping is performed on the output factor of the butterfly computation through the transposition circuit **13$_2$-8**, memory address change is performed on the position-swapped factor through the memory transposition circuit **13$_2$-9**, factor position swapping is performed on the address-changed factor through the transposition circuit **13$_2$-10**, the butterfly computation is performed on the position-swapped factor through the stage conversion circuit **13$_2$-11** of log $^{d}\textcircled{N}$ sub-stages (d is 3), the multiplication computation is performed on the output factor of the butterfly computation and the corresponding twiddle factor through the second twiddle circuit **13$_2$-12**, and the multiplication computation is performed on the output factor of the multiplication computation and the corresponding twiddle factor through the first twiddle circuit **13$_2$-13**. That is, three-dimensional multi-dimensional fast number theoretic transform is performed on the input matrix through the computing circuits **13** (that is, the two first twiddle circuits **13$_2$-1** and **13$_2$-13**, the two second twiddle circuits **13$_2$-2** and **13$_2$-12**, the four transposition circuits **13$_2$-4**, **13$_2$-6**, **13$_2$-8**, and **13$_2$-10**, and the two memory transposition circuits **13$_2$-5** and **13$_2$-9**).

[0077] FIG. **9**A to FIG. **9**F are schematic diagrams illustrating three-dimensional fast number theoretic transform according to an embodiment of the disclosure. Please refer to FIG. **9**A to FIG. **9**F. $\omega^{N_1N_2n_3}$, $(\omega^{2k_3})^{N_1N_2n_3}$, $(\omega^{2k_3+1})^{N_1n_2+n_1}$, $(\omega^{2k_2})^{N_3N_1n_2}$, $(\omega^{2N_3k_2})^{N_1n_2+n_1}$, and $(\omega^{2k_1})^{N_3N_2n_1}$ are the twiddle factors used in the multiplication computations. The data conversion circuit converts 1×16 input data into two 4×2 input matrices, such as

$$\begin{bmatrix} 11 & 03 \\ 10 & 02 \\ 09 & 01 \\ 08 & 00 \end{bmatrix} \text{ and } \begin{bmatrix} 15 & 07 \\ 14 & 06 \\ 13 & 05 \\ 12 & 04 \end{bmatrix}.$$

That is, one dimension is added to store or compute another two-dimensional matrix. In Step S**901**, the multiplication computation is performed on the twiddle factor $\omega^{N_2N_2n_3}$ is multiplied, such as

$$\begin{bmatrix} 11 & 03 \\ 10 & 02 \\ 09 & 01 \\ 08 & 00 \end{bmatrix} \times \begin{bmatrix} 22 \\ 1 \end{bmatrix}.$$

In Step S**902**, the multiplication computation is performed on the twiddle factor $(\omega^{2k_3})^{N_1N_2n_3}$. In Step S**903**, the multiplication computation is performed on the twiddle factor $(\omega^{2k_3+1})^{N_1n_2+n_1}$. In Step S**904**, the multiplication computation is performed on the twiddle factor $(\omega^{2k_2})^{N_3N_1n_2}$. In Step S**905**, the multiplication computation is performed on the twiddle factor $(\omega^{2N_3k_2})^{N_1n_2+n_1}$. In Step S**906**, the multiplication computation is performed on the twiddle factor $(\omega^{2k_1})^{N_3N_2n_1}$. The "twiddle" processing in the drawings is, for example, the sub-computation shown in FIG. **4**B, that is, the multiplication computation is performed on the corresponding twiddle factor. In addition, the "transposition" processing in the drawings is, for example, the sub-computation shown in FIG. **4**C, that is, factor position swapping. Finally, the output matrices

$$\begin{bmatrix} 78 & 34 \\ 09 & 03 \\ 73 & 89 \\ 74 & 96 \end{bmatrix} \text{ and } \begin{bmatrix} 75 & 10 \\ 36 & 73 \\ 41 & 67 \\ 88 & 87 \end{bmatrix},$$

and the values are the same as the values in the output vector of FIG. **5**C and the output matrix of FIG. **6**D (the difference is only in the positions of the matrix).

[0078] In an embodiment, the twiddle factors may be prestored in the memory **11** for subsequent computations. In another embodiment, some or all of the twiddle factors may be generated on the fly or dynamically. For example, the generated twiddle factors may be computed on the fly using the multiplication factor and the starting twiddle factor. At this time, the amount of data required to be stored is O(n^(½)).

[0079] FIG. **10** is a system architecture diagram of four-dimensional fast number theoretic transform according to an embodiment of the disclosure. Please refer to FIG. **10**, which shows a four-dimensional pipelined multi-dimensional fast number theoretic transform architecture. For a four-dimensional computation (d of Equation (2) is 4), the mathematical expression of multi-dimensional fast number theoretic transform is:

$$X_{N_4N_3N_2k_1+N_4N_3k_2+N_4k_3+k_4} = \sum_{n_1=0}^{N_1-1}\sum_{n_2=0}^{N_2-1}\sum_{n_3=0}^{N_3-1}\sum_{n_4=0}^{N_4-1} x_{N_1N_2N_3n_4+N_1N_2n_3+N_1n_2+n_1} \left(\omega^{2(N_4N_3N_2k_1+N_4N_3k_2+N_4k_3+k_4)+1}\right)^{N_1N_2N_3n_4+N_1N_2n_3+N_1n_2+n_1} = \sum_{n_2=0}^{N_2-1}\left(\sum_{n_3=0}^{N_3-1}\left(\left(\sum_{n_4=0}^{N_4-1} x_{N_4N_3N_2k_1+N_4N_3k_2+N_4k_3+k_4}\omega^{N_1N_2N_3n_4}\left(\omega^{2k_4}\right)^{N_1N_2N_3n_4}\right)\left(\omega^{2k_4+1}\right)^{N_1N_2n_3+N_1n_2+n_1}\left(\omega^{2k_3}\right)^{N_4N_2N_1n_3}\right)\right)\left(\omega^{2N_4k_3}\right)^{N_1N_2n_3+N_1n_2+n_1}\left(\omega^{2k_2}\right)^{N_4N_3N_1n_3}\left(\omega^{2N_4N_3k_2}\right)^{N_1n_2+n_1}\left(\omega^{2k_1}\right)^{N_4N_3N_2n_1} \quad (5)$$

[0080] where $X_{N_4N_3N_2k_1+N_4N_3k_2+N_4k_3+k_4}$ is the output factor of $(N_4N_3N_2k_1+N_4N_3k_2+N_4k_3+k_4)$-th multi-dimensional fast number theoretic transform, $x_{N_4N_3N_2k_1+N_4N_3k_2+N_4k_3+k_4}$ is the $(N_4N_3N_2k_1+N_4N_3k_2+N_4k_3+k_4)$-th input matrix, ω is the twiddle factor, $N_1 \cdot N_2 \cdot N_3 \cdot N_4$ is N^4, and N is the degree of the polynomial.

[0081] Equation (5) may be allocated to the computing circuits **13** for implementation. The computing circuits **13** include one or more first twiddle circuits, one or more stage conversion circuits, one or more transposition circuits, and one or more memory transposition circuits. Taking FIG. **10** as an example, the computing circuits **13** with such an architecture include four first twiddle circuits **13$_3$-1**, **13$_3$-3**, **13$_3$-8**, and **13$_3$-13**, four stage conversion circuits **13$_3$-2**, **13$_3$-7**, **13$_3$-12**, and **13$_3$-16**, five transposition circuits **13$_3$-4**, **13$_3$-6**, **13$_3$-9**, **13$_3$-11**, and **13$_3$-14**, and three memory transposition circuits **13$_3$-5**, **13$_3$-10**, and **13$_3$-15**. The order of execution of the computing circuits **13** is the first twiddle

circuit **13$_3$-1**, the stage conversion circuit **13$_3$-2**, the first twiddle circuit **13$_3$-3**, the transposition circuit **13$_3$-4**, the memory transposition circuit **13$_3$-5**, the transposition circuit **13$_3$-6**, the stage conversion circuit **13$_3$-7**, the first twiddle circuit **13$_3$-8**, the transposition circuit **13$_3$-9**, the memory transposition circuit **13$_3$-10**, the transposition circuit **13$_3$-11**, the stage conversion circuit **13$_3$-12**, the first twiddle circuit **13$_3$-13**, the transposition circuit **13$_3$-14**, the memory transposition circuit **13$_3$-15**, and the stage conversion circuit **13$_3$-16**.

[0082] In an embodiment, the sub-computations of the first twiddle circuits **13$_3$-1**, **13$_3$-3**, **13$_3$-8**, and **13$_3$-13** are the multiplication computations of the twiddle factors, and reference may be made to the description of FIG. **4**B.

[0083] In an embodiment, the sub-computations of the stage conversion circuits **13$_3$-2**, **13$_3$-7**, **13$_3$-12**, and **13$_3$-16** are the butterfly computations, and reference may be made to the description of FIG. **4**A.

[0084] In an embodiment, the sub-computations of the transposition circuits **13$_3$-4**, **13$_3$-6**, **13$_3$-9**, **13$_3$-11**, and **13$_3$-14** are factor position swapping, and reference may be made to the description of FIG. **4**C.

[0085] In an embodiment, the sub-computations of the memory transposition circuits **13$_3$-5**, **13$_3$-10**, and **13$_3$-15** are memory address change, and reference may be made to the description of FIG. **8**B.

[0086] Please refer to FIG. **10**. The data conversion circuit **12** converts the input data into the input matrix according to the target dimension. The input data is the factors of the polynomial and is in vector form. The input data is, for example, data related to fully homomorphic encrypted ciphertext or plaintext. In addition, the target dimension is greater than one dimension (four dimensions in the embodiment). Therefore, the data conversion circuit **12** vectorizes the factors of the polynomial into the input data, and converts the input data into the two-dimensional input matrix. The column number or the row number of the input matrix is $\sqrt[d]{N}$, such as $\sqrt[d]{N}$.

[0087] In response to the target dimension being four-dimensional, the mathematical expression of multi-dimensional fast number theoretic transform is determined as Equation (5). Then, the multiplication computation is performed on the input matrix and the corresponding twiddle factor through the first twiddle circuit **13$_3$-1**, the butterfly computation is performed on the output factor of the multiplication computation through the stage conversion circuit **13$_3$-2** of log $\sqrt[d]{N}$ sub-stages (d is 4), the multiplication computation is performed on the output factor of the butterfly computation and the corresponding twiddle factor through the first twiddle circuit **13$_3$-3**, factor position swapping is performed on the output factor of the multiplication computation through the transposition circuit **13$_3$-4**, memory address change is performed on the position-swapped factor through the memory transposition circuit **13$_3$-5**, memory address change is performed on the position-swapped factor through the transposition circuit **13$_3$-6**, the butterfly computation is performed on the address-changed factor through the stage conversion circuit **13$_3$-7** of log $\sqrt[4]{N}$ sub-stages, the multiplication computation with the corresponding twiddle factor is performed through the first twiddle circuit **13$_3$-8**, factor position swapping is performed on the output factor of the multiplication computation through the transposition circuit **13$_3$-9**, memory address change is performed on the position-swapped factor through the memory transposition circuit **13$_3$-10**, factor position swapping is performed on the address-changed factor through the transposition circuit **13$_3$-11**, the butterfly computation is performed on the position-swapped factor through the stage conversion circuit **13$_3$-12** of log $\sqrt[4]{N}$ sub-stages, the multiplication computation is performed on the output factor of the butterfly computation and the corresponding twiddle factor through the first twiddle circuit **13$_3$-13**, factor position swapping is performed on the output factor of the multiplication computation through the transposition circuit **13$_3$-14**, memory address change is performed on the position-swapped factor through the memory transposition circuit **13$_3$-15**, and the butterfly computation is performed on the address-changed factor through the stage conversion circuit **13$_3$-16** of log $\sqrt[4]{N}$ sub-stages.

[0088] FIG. **11**A to FIG. **11**F are schematic diagrams illustrating four-dimensional fast number theoretic transform according to an embodiment of the disclosure. Please refer to FIG. **11**A to FIG. **11**F. $\omega^{N_1N_2N_3n_4}$, $(\omega^{2k_4})^{N_1N_2N_3n_4}$, $(\omega^{2k_4+1})^{N_1N_2n_3+N_1n_2+n_1}$, $(\omega^{2k_3})^{N_4N_2N_1n_3}$, $(\omega^{2N_4k_3})^{N_1N_2n_3+N_1n_2+n_1}$, $(\omega^{2k_2})^{N_4N_3N_1n_3}$, $(\omega^{2N_4N_3k_2})^{N_1n_2+n_1}$, and $(\omega^{2k_1})^{N_4N_3N_2n_1}$ are the twiddle factors used in the multiplication computations. The data conversion circuit converts 1×16 input data into four 2×2 input matrices, such as

$$\begin{bmatrix}12 & 04\\ 08 & 00\end{bmatrix},\begin{bmatrix}13 & 05\\ 09 & 01\end{bmatrix},\begin{bmatrix}14 & 06\\ 10 & 02\end{bmatrix},\text{ and }\begin{bmatrix}15 & 07\\ 11 & 03\end{bmatrix}.$$

That is, two dimensions are added to store or compute another two two-dimensional matrices. In Step **S1101**, the multiplication computation is performed on the twiddle factor $\omega^{N_1N_2n_3n_4}$, such as

$$\begin{bmatrix}12 & 04\\ 08 & 00\end{bmatrix}\times\begin{bmatrix}22\\ 1\end{bmatrix}.$$

In Step **S1102**, the multiplication computation is performed on the twiddle factor $(\omega^{2k_4})^{N_1N_2N_3n_4}$. In Step **S1103**, the multiplication computation is performed on the twiddle factor $(\omega^{2k_4+1})^{N_1N_2n_3+N_1n_2+n_1}$. In Step **S1104**, the multiplication computation is performed on the twiddle factor $(\omega^{2k_3})^{N_4N_2N_1n_3}$. In Step **S1105**, the multiplication computation is performed on the twiddle factor $(\omega^{2N_4k_3})^{N_1N_2n_3+N_1n_2+n_1}$. In Step **S1106**, the multiplication computation is performed on the twiddle factor $(\omega^{2k_2})^{N_4N_3N_1n_3}$. In Step **S1107**, the multiplication computation is performed on the twiddle factor $(\omega^{2N_4N_3k_2})^{N_1n_2+n_1}$. In Step **S1108**, the multiplication computation is performed on the twiddle factor $(\omega^{2k_1})^{N_4N_3N_2n_1}$. The "twiddle" processing in the drawings is, for example, the sub-computation shown in FIG. **4**B, that is, the multiplication computation is performed on the corresponding twiddle factor. In addition, the "transposition" processing in the drawings is, for example, the sub-computation shown in FIG. **4**C, that is, factor position swapping. Finally, the output matrix is

$$\begin{bmatrix}88 & 87\\ 74 & 96\end{bmatrix},\begin{bmatrix}41 & 67\\ 73 & 89\end{bmatrix},\begin{bmatrix}36 & 73\\ 09 & 03\end{bmatrix},\text{ and }\begin{bmatrix}75 & 10\\ 78 & 34\end{bmatrix},$$

and the values are the same as the values in the output vector of FIG. **5**C and the output matrices of FIG. **6**D and FIG. **9**F (the difference is only in the positions of the matrix).

[0089] In an embodiment, the twiddle factors may be prestored in the memory **11** for subsequent computations. In another embodiment, some or all of the twiddle factors may be generated on the fly or dynamically. For example, the resulting twiddle factor may be computed on the fly using the multiplication factor and the starting twiddle factor. At this time, the amount of data required to be stored is O(n^(⅓)).

[0090] FIG. **12** is a system architecture diagram of non-pipelined fast number theoretic transform according to an embodiment of the disclosure. Please refer to FIG. **12**, which shows a non-pipelined fast number theoretic transform architecture. Such an architecture may be implemented through the computing circuits **13**. The computing circuits **13** include one or more stage conversion circuits and one or more memory circuits. Taking FIG. **12** as an example, the computing circuits **13** with such an architecture include a stage conversion circuit $\mathbf{13}_4$-**1** and a memory circuit $\mathbf{13}_4$-**2**. The order of execution of the computing circuits **13** is the stage conversion circuit $\mathbf{13}_4$-**1** and the memory circuit $\mathbf{13}_4$-**2**.

[0091] In an embodiment, the sub-computation of the stage conversion circuit $\mathbf{13}_4$-**1** is the butterfly computation, and reference may be made to the description of FIG. **4**A.

[0092] In an embodiment, the sub-computation of the memory circuit $\mathbf{13}_4$-**2** is to store data to one or more memories **11**. FIG. **13** is a schematic diagram illustrating data storage according to an embodiment of the disclosure. Please refer to FIG. **13**. Data is accessed from the memory **11**, the butterfly computation is performed on the accessed data, and a result of the butterfly computation is stored in the memory **11**. It should be noted that the width in the butterfly computation may be changed according to actual requirements.

[0093] Please refer to FIG. **12**. The data conversion circuit **12** converts the input data into the input matrix according to the target dimension. The input data is the factors of the polynomial and is in vector form. The input data is, for example, data related to fully homomorphic encrypted ciphertext or plaintext. In addition, the target dimension is greater than one dimension (two dimensions in the embodiment). Therefore, the data conversion circuit **12** vectorizes the factors of the polynomial into the input data, and converts the input data into the two-dimensional input matrix. At this time, the column number or the row number of the input matrix may be adjusted according to actual requirements.

[0094] In response to a non-pipelined computation, the butterfly computation is performed through the stage conversion circuit $\mathbf{13}_4$-**1** of one sub-stage, and the output factor of the butterfly computation is stored in the memory **11** through the memory circuit $\mathbf{13}_4$-**2**.

[0095] FIG. **14**A is a schematic diagram illustrating forward fast number theoretic transform according to an embodiment of the disclosure, and FIG. **14**B is a schematic diagram illustrating non-pipelined fast number theoretic transform according to an embodiment of the disclosure. Please refer to FIG. **14**B. The data conversion circuit converts 1×16 input data into a 4×4 input matrix, such as

$$\begin{bmatrix} 15 & 11 & 07 & 03 \\ 14 & 10 & 06 & 02 \\ 13 & 09 & 05 & 01 \\ 12 & 08 & 04 & 00 \end{bmatrix}.$$

However, the column number or the row number of the input matrix may still be changed according to actual requirements. The "storage" processing in the drawings is, for example, the sub-computation shown in FIG. **13**, that is, the butterfly computation is performed and a computation result is stored. The output matrix is

$$\begin{bmatrix} 18 & 36 & 08 & 67 \\ 13 & 51 & 72 & 32 \\ 08 & 67 & 55 & 22 \\ 27 & 60 & 49 & 08 \end{bmatrix}$$

(corresponding to a result of forward fast number theoretic transform shown in FIG. **14**A, and the difference is only in the positions of the matrix).

[0096] In addition to two-dimensional, three-dimensional, and four-dimensional multi-dimensional fast number theoretic transform above, higher-dimensional multi-dimensional fast number theoretic transform may also be extended to based on Equation (2). In some application scenarios, higher-dimensional hardware architectures have deeper pipelined depths and longer computing delay, but smaller data throughput for the same number of times. However, higher-dimensional hardware architectures require fewer multipliers and require smaller amounts of data of twiddle factors to be stored. In other words, lower-dimensional hardware architectures have shallower pipelined depths and shorter computing delay, but greater data throughput for the same number of times. However, lower-dimensional hardware architectures require more multipliers and require greater amounts of data of twiddle factors to be stored.

[0097] For example, Table (1) shows the correspondence between the number of multipliers, the number of adders, the number of block memories, and the delay of the target dimension being respectively two-dimensional, three-dimensional, and four-dimensional under a condition that an experimental target is N=65536.

TABLE 1

N = 65536	Number of multipliers	Number of adders	Number of block memories	Delay
Two-dimensional (256 × 256)	2560	4096	64	274
Three-dimensional (32 × 32 × 65)	432	608	240	8337
Four-dimensional (16 × 16 × 16 × 16)	256	256	168	12388

In terms of multiplier resource usage, the architecture of the target dimension being three-dimensional is 1.7 times the architecture of the target dimension being four-dimensional, but the delay is 0.67 times.

[0098] For another example, Table (2) shows the correspondence between the number of multipliers, the number of adders, the number of block memories, and the delay of the target dimension being respectively two-dimensional, three-

dimensional, and four-dimensional under a condition that the experimental target is N=16384.

TABLE 2

N = 16384	Number of multipliers	Number of adders	Number of block memories	Delay
Two-dimensional (128 × 128)	1152	1792	32	144
Three-dimensional (32 × 32 × 32)	304	608	240	2193
Four-dimensional (16 × 16 × 16 × 4)	256	256	168	3172

In terms of multiplier resource usage, the architecture of the target dimension being three-dimensional is 1.3 times the architecture of the target dimension being four-dimensional, but the delay is 0.69 times.

[0099] In an embodiment, the processing system **10** includes multiple first computing circuits and multiple second computing circuits. The first computing circuits include the computing circuits **13** for multi-dimensional fast number theoretic transform of a first dimension, the second computing circuits include the computing circuits **13** for multi-dimensional fast number theoretic transform of a second dimension, and the first dimension is different from the second dimension. For example, the first computing circuits include the two stage conversion circuits $\mathbf{13_1}$**-1** and $\mathbf{13_1}$**-4**, the first twiddle circuit $\mathbf{13_1}$**-2**, and the transposition circuit $\mathbf{13_1}$**-3** (that is, the first dimension is two-dimensional) shown in FIG. **3**, and the second computing circuits include the two first twiddle circuits $\mathbf{13_2}$**-1** and $\mathbf{13_2}$**-13**, the two second twiddle circuits $\mathbf{13_2}$**-2** and $\mathbf{13_2}$**-12**, the three stage conversion circuits $\mathbf{13_2}$**-3**, $\mathbf{13_2}$**-7**, and $\mathbf{13_2}$**-11**, the four transposition circuits $\mathbf{13_2}$**-4**, $\mathbf{13_2}$**-6**, $\mathbf{13_2}$**-8**, and $\mathbf{13_2}$**-10**, and the two memory transposition circuits $\mathbf{13_2}$**-5** and $\mathbf{13_2}$**-9** (that is, the second dimension is three-dimensional) as shown in FIG. **7**.

[0100] The data conversion circuit **12** may convert the input data into the input matrix that conforms to the first computing circuits or the second computing circuits according to the target dimension. For example, if the target dimension is two-dimensional, the input data is converted into the input matrix with the column number $\sqrt{N}$; and if the target dimension is four-dimensional, the input data is converted into the input matrix with the column number

$$\sqrt[d]{N}$$

(d is 4).

[0101] In an embodiment, the data conversion circuit **12** may select one of the first computing circuit and the second computing circuit to perform fast number theoretic transform according to requirements (for example, hardware limitations or delay limitations). For example, if the delay limitation is short, a set of the computing circuits **13** with a smaller target dimension is selected to perform fast number theoretic transform.

[0102] In some embodiments, the processing system **10** may include more computing circuits **13** corresponding to different dimensions. For example, the processing system **10** includes the computing circuits **13** corresponding to two dimensions, three dimensions, and four dimensions.

[0103] FIG. **15** is a flowchart of an encryption method according to an embodiment of the disclosure. Please refer to FIG. **15**. The computing circuits **13** are used to perform the fast number theoretic transform calculation, such as two-dimensional fast number theoretic transform shown in FIG. **3**, FIG. **4**A to FIG. **4**C, and FIG. **6**A to FIG. **6**D, one-dimensional fast number theoretic transform shown in FIG. **5**A to FIG. **5**C, three-dimensional fast number theoretic transform shown in FIG. **7**, FIG. **8**A to FIG. **8**B, and FIG. **9**A to FIG. **9**F, four-dimensional fast number theoretic transform shown in FIG. **10** and FIG. **11**A to FIG. **11**F, or non-pipelined fast number theoretic transform shown in FIG. **12**, FIG. **13**, and FIG. **14**A and FIG. **14**B, on the polynomial (Step **S1501**). The computing circuits **13** perform the matrix multiplication calculation on the polynomial through the fast number theoretic transform calculation (Step **S1502**). In an embodiment, the elements of the input matrix are the factors of the polynomial used for encryption, such as the factors of the polynomial related to ciphertext or plaintext used for fully homomorphic encryption. In an embodiment, the elements of the input matrix are respectively the same as the elements in the input data (one-to-one element correspondence). The input data is also the factors of the same polynomial and is in vector form. In an embodiment, the fast number theoretic transform is calculated as multi-dimensional fast number theoretic transform of the target dimension. The target dimension may be determined. The target dimension is greater than one dimension. For example, the data conversion circuit **12** determines whether the target dimension is two-dimensional, three-dimensional, or four-dimensional. The input data is converted into the input matrix according to the target dimension. For example, in response to the target dimension being three-dimensional, the data conversion circuit **12** converts the vector input data into the two two-dimensional input matrices. Then, multi-dimensional fast number theoretic transform of the target dimension is performed on the input matrix through the computing circuits **13**. Multi-dimensional fast number theoretic transform converts matrix-vector multiplication into a multi-stage computation architecture of the matrix multiplication calculation through multiple sub-computations, one computing circuit **13** performs one sub-computation, and each sub-computation is located at one stage of the multi-stage computation architecture.

[0104] The implementation details of each step in FIG. **15** have been described in detail in the foregoing embodiments and implementations, and will not be described again here.

[0105] In some application scenarios, in addition to a fully homomorphic encryption system, multi-dimensional fast number theoretic transform of the embodiments of the disclosure is also suitable for various lattice cryptography and various applications related to polynomial calculations.

[0106] In some application scenarios, a field programmable gate array (FPGAs) has powerful parallel computing ability and can handle encryption and computing requirements of multiple data blocks at the same time. Implementing the multi-stage hardware architecture of the embodiments of the disclosure through the FPGA may make full use of the parallel computing ability of the FPGA, thereby implementing high-efficiency data processing. However, according to different design requirements, the processing system **10** is not limited to the implementation form of the FPGA.

[0107] In some application scenarios, fully homomorphic encryption can ensure that data during a computation process remains encrypted, thereby ensuring the security of the data. The embodiments of the disclosure can strengthen the security protection and ensure that the data is not exposed to an unauthorized environment at any time.

[0108] In some application scenarios, in big data scenarios, the amount of data is huge and needs to be processed in a timely manner, which puts higher requirements on the traditional fully homomorphic encryption technology. The embodiments of the disclosure can provide the high-efficiency fast number theoretic transform computation and the parallel computing ability of the processing system **10**, thereby meeting the processing requirements for large-scale data.

[0109] In some application scenarios, the multiplication computation related to the polynomial in fully homomorphic encryption may be implemented through the processing system **10** of the embodiments of the disclosure, such as an encryption and/or decryption computation on plaintext or ciphertext.

[0110] In summary, in the processing system and method related to encryption of the embodiments of the disclosure, the sub-computations in multi-dimensional fast number theoretic transform are respectively implemented through the corresponding computing circuits, and the pipelined and non-pipelined architectures are provided. In this way, the computing efficiency of fast number theoretic transform can be improved, and fully homomorphic encryption or other lattice-based encryption algorithms are also suitable.

[0111] Although the disclosure has been disclosed in the above embodiments, the embodiments are not intended to limit the disclosure. Persons skilled in the art may make some changes and modifications without departing from the spirit and scope of the disclosure. Therefore, the protection scope of the disclosure shall be defined by the appended claims.

What is claimed is:

1. A processing system related to encryption, comprising:

a plurality of computing circuits, performing a fast number theoretic transform (NTT) calculation on a polynomial, comprising:

performing a matrix multiplication calculation on the polynomial through the fast number theoretic transform calculation.

2. The processing system related to encryption according to claim **1**, further comprising:

at least one memory, coupled to the computing circuits and configured to store data;

a data conversion circuit, coupled to the at least one memory and the computing circuits, and configured to convert input data into an input matrix according to a target dimension, wherein the input data is factors of the polynomial and is in vector form, the target dimension is greater than one dimension, the fast number theoretic transform calculation is multi-dimensional fast number theoretic transform, and

the computing circuits are further configured to:

perform the multi-dimensional fast number theoretic transform of the target dimension on the input matrix, wherein the multi-dimensional fast number theoretic transform converts matrix-vector multiplication into a multi-stage computation architecture of the matrix multiplication calculation through a plurality of sub-calculations, one computing circuit performs one sub-calculation, and each of the sub-calculations is located at one stage of the multi-stage computation architecture.

3. The processing system related to encryption according to claim **2**, wherein:

in response to the target dimension being two-dimensional, a mathematical expression of the multi-dimensional fast number theoretic transform is:

$$X_{N_2k_1+k_2}=\sum_{n_1=0}^{N_1-1}\left(\left(\sum_{n_2=0}^{N_2-1}x_{N_1n_2+n_1}\left(\omega^{2k_2+1}\right)^{N_1n_2}\right)\omega^{n_1(2k_2+1-N_2)}\right)\left(\omega^{2k_1+1}\right)^{N_2n_1}$$

where $X_{N_2k_1+k_2}$ is an output factor of the $(N_2k_1+k_2)$-th multi-dimensional fast number theoretic transform, $x_{N_1n_2+n_1}$ is the $(N_1n_2+n_1)$-th input matrix, ω is a twiddle factor, $N_1 \cdot N_2$ is N^2, and N is a degree of the polynomial.

4. The processing system related to encryption according to claim **3**, wherein the computing circuits comprise at least one stage conversion circuit, a first twiddle circuit, and a transposition circuit, a sub-computation of the at least one stage conversion circuit is a butterfly computation, a sub-computation of the first twiddle circuit is a multiplication computation of the twiddle factor, and a sub-computation of the transposition circuit is factor position swapping.

5. The processing system related to encryption according to claim **2**, wherein:

in response to the target dimension being three-dimensional, a mathematical expression of the multi-dimensional fast number theoretic transform is:

$$X_{N_3N_2k_1+N_3k_2+k_3}=\sum_{n_1=0}^{N_1-1}\left(\sum_{n_2=0}^{N_2-1}\left(\left(\sum_{n_3=0}^{N_3-1}x_{N_1N_2n_3+N_1n_2+n_1}\omega^{N_1N_2n_3}\left(\omega^{2k_3}\right)^{N_1N_2n_3}\right)\left(\omega^{2k_3+1}\right)^{N_1N_2+n_1}\left(\omega^{2k_2}\right)^{N_3N_1n_2}\right)\right)\left(\omega^{2N_3k_2}\right)^{N_1n_2+n_1}\left(\omega^{2k_1}\right)^{N_3N_2n_1}$$

where $X_{N_3N_2k_1+N_3k_2+k_3}$ is an output factor of the $(N_3N_2k_1+N_3k_2+k_3)$-th multi-dimensional fast number theoretic transform, $x_{N_1N_2n_3+N_1n_2+n_1}$ is the $(N_1N_2n_3+N_1n_2+n_1)$-th input matrix, ω is a twiddle factor, $N_1 \cdot N_2 \cdot N_3$ is N^3, and N is a degree of the polynomial.

6. The processing system related to encryption according to claim **5**, wherein the computing circuits comprise at least one first twiddle circuit, at least one second twiddle circuit, at least one stage conversion circuit, a transposition circuit, and at least one memory transposition circuit, sub-computations of the at least one first twiddle circuit and the at least one second twiddle circuit are multiplication computations of the twiddle factor, a sub-computation of the at least one stage conversion circuit is a butterfly computation, a sub-computation of the transposition circuit is factor position swapping, and a sub-computation of the at least one memory transposition circuit is memory address change.

7. The processing system related to encryption according to claim **2**, wherein:

in response to the target dimension being four-dimensional, a mathematical expression of the multi-dimensional fast number theoretic transform is:

$$X_{N_4N_3N_2k_1+N_4N_3k_2+N_4k_3+k_4} = \sum_{n_2=0}^{N_2-1}\left(\sum_{n_3=0}^{N_3-1}\left(\left(\sum_{n_4=0}^{N_4-1} x_{N_4N_3N_2k_1+N_4N_3k_2+N_4k_3+k_4}\omega^{N_1N_2N_3n_4}\left(\omega^{2k_4}\right)^{N_1N_2N_3n_4}\right)\left(\omega^{2k_4+1}\right)^{N_1N_2n_3+N_1n_2+n_1}\left(\omega^{2k_3}\right)^{N_4N_2N_1n_3}\right)\right)\left(\omega^{2N_4k_3}\right)^{N_1N_2n_3+N_1n_2+n_1}\left(\omega^{2k_2}\right)^{N_4N_3N_1n_3}\left(\omega^{2N_4N_3k_2}\right)^{N_1n_2+n_1}\left(\omega^{2k_1}\right)^{N_4N_3N_2n_1}$$

where $X_{N_4N_3N_2k_1+N_4N_3k_2+N_4k_3+k_4}$ is an output factor of the $(N_4N_3N_2k_1+N_4N_3k_2+N_4k_3+k_4)$-th multi-dimensional fast number theoretic transform, $x_{N_4N_3N_2k_1+N_4N_3k_2+N_4k_3+k_4}$ is the $(N_4N_3N_2k_1+N_4N_3k_2+N_4k_3+k_4)$-th input matrix, ω is a twiddle factor, $N_1 \cdot N_2 \cdot N_3 \cdot N_4$ is N^4, and N is a degree of the polynomial.

8. The processing system related to encryption according to claim **7**, wherein the computing circuits comprise at least one first twiddle circuit, at least one stage conversion circuit, at least one transposition circuit, and at least one memory transposition circuit, a sub-computation of the at least one first twiddle circuit is a multiplication computation of the twiddle factor, a sub-computation of the at least one stage conversion circuit is a butterfly computation, a sub-computation of the at least one transposition circuit is factor position swapping, and a sub-computation of the at least one memory transposition circuit is memory address change.

9. The processing system related to encryption according to claim **2**, wherein:

a mathematical expression of the multi-dimensional fast number theoretic transform is:

$$X_{k_1,k_2,\ldots,k_d} = \sum_{n_1=0}^{N_1-1}\left(\omega_{N_1}^{k_1n_1}\sum_{n_2=0}^{N_2-1}\omega_{N_2}^{k_2n_2}\cdots\sum_{n_d=0}^{N_d-1}\omega_{N_d}^{k_dn_d}\cdot x_{n_1,n_2,\ldots,n_d}\right)$$

where $X_{k_1,k_2,\ldots,k_d}$ is an output factor of $(k_1, k_2, \ldots, k_d)$-th multi-dimensional fast number theoretic transform, $x_{n_1,n_2,\ldots,n_d}$ is $(n_1, n_2, \ldots, n_d)$-th input matrix, ω is a twiddle factor, $N_1 \cdot N_2 \cdot \ldots \cdot N_d$ is N^d, and N is a degree of the polynomial.

10. The processing system related to encryption according to claim **2**, wherein the computing circuits comprise a stage conversion circuit and a memory circuit, a sub-computation of the stage conversion circuit is a butterfly computation, and a sub-computation of the memory circuit is to store data to the at least one memory.

11. A method related to encryption, comprising:

performing a fast number theoretic transform calculation on a polynomial through a plurality of computing circuits, comprising:

performing a matrix multiplication calculation on the polynomial through the fast number theoretic transform calculation.

12. The method related to encryption according to claim **11**, wherein the fast number theoretic transform calculation is multi-dimensional fast number theoretic transform, a plurality of elements of an input matrix are factors of the polynomial, and performing the matrix multiplication calculation on the polynomial through the fast number theoretic transform calculation comprises:

performing the multi-dimensional fast number theoretic transform of a target dimension on the input matrix through the computing circuits, wherein the target dimension is greater than one dimension, the multi-dimensional fast number theoretic transform converts a matrix-vector multiplication into a multi-stage computation architecture of the matrix multiplication calculation through a plurality of sub-calculations, one computing circuit performs one sub-calculation, and each of the sub-calculations is located at one stage of the multi-stage computation architecture.

13. The method related to encryption according to claim **12**, wherein performing the multi-dimensional fast number theoretic transform of the target dimension on the input matrix through the computing circuits comprises:

in response to the target dimension being two-dimensional, determining a mathematical expression of the multi-dimensional fast number theoretic transform as:

$$X_{N_2k_1+k_2} = \sum_{n_1=0}^{N_1-1}\left(\left(\sum_{n_2=0}^{N_2-1} x_{N_1n_2+n_1}\left(\omega^{2k_2+1}\right)^{N_1n_2}\right)\omega^{n_1(2k_2+1-N_2)}\right)\left(\omega^{2k_1+1}\right)^{N_2n_1}$$

where $X_{N_2k_1+k_2}$ is an output factor of the $(N_2k_1+k_2)$-th multi-dimensional fast number theoretic transform, $x_{N_1n_2+n_1}$ is the $(N_1n_2+n_1)$-th input matrix, ω is a twiddle factor, $N_1 \cdot N_2$ is N^2, and N is a degree of the polynomial.

14. The method related to encryption according to claim **13**, wherein the computing circuits comprise at least one stage conversion circuit, a first twiddle circuit, and a transposition circuit, the method further comprising:

performing a butterfly computation through the at least one stage conversion circuit;

performing a multiplication computation of the twiddle factor through the first twiddle circuit; and

performing factor position swapping through the transposition circuit.

15. The method related to encryption according to claim **12**, wherein performing the multi-dimensional fast number theoretic transform of the target dimension on the input matrix through the computing circuits comprises:

in response to the target dimension being three-dimensional, determining a mathematical expression of the multi-dimensional fast number theoretic transform as:

$$X_{N_3N_2k_1+N_3k_2+k_3} = \sum_{n_1=0}^{N_1-1}\left(\sum_{n_2=0}^{N_2-1}\left(\left(\sum_{n_3=0}^{N_3-1} x_{N_1N_2n_3+N_1n_2+n_1}\omega^{N_1N_2n_3}\left(\omega^{2k_3}\right)^{N_1N_2n_3}\right)\left(\omega^{2k_3+1}\right)^{N_1N_2+n_1}\left(\omega^{2k_2}\right)^{N_3N_1n_2}\right)\right)\left(\omega^{2N_3k_2}\right)^{N_1n_2+n_1}\left(\omega^{2k_1}\right)^{N_3N_2n_1}$$

where $X_{N_3N_2k_1+N_3k_2+k_3}$ is an output factor of the $(N_3N_2k_1+N_3k_2+k_3)$-th multi-dimensional fast number theoretic transform, $x_{N_1N_2n_3+N_1n_2+n_1}$ is the $(N_1N_2n_3+N_1n_2+n_1)$-th input matrix, ω is a twiddle factor, $N_1 \cdot N_2 \cdot N_3$ is N^3, and N is a degree of the polynomial.

16. The method related to encryption according to claim **15**, wherein the computing circuits comprise at least one first twiddle circuit, at least one second twiddle circuit, at least one stage conversion circuit, a transposition circuit, and at least one memory transposition circuit, the method further comprising:

performing a multiplication computation of the twiddle factor through the at least one first twiddle circuit and the at least one second twiddle circuit;

performing a butterfly computation through the at least one stage conversion circuit;

performing factor position swapping through the transposition circuit; and

performing memory address change through the memory transposition circuit.

17. The method related to encryption according to claim **12**, wherein performing the multi-dimensional fast number theoretic transform of the target dimension on the input matrix through the computing circuits comprises:

in response to the target dimension being four-dimensional, determining a mathematical expression of the multi-dimensional fast number theoretic transform as:

$$X_{N_4N_3N_2k_1+N_4N_3k_2+N_4k_3+k_4} = \sum_{n_2=0}^{N_2-1}\left(\sum_{n_3=0}^{N_3-1}\left(\left(\sum_{n_4=0}^{N_4-1} x_{N_4N_3N_2k_1+N_4N_3k_2+N_4k_3+k_4}\omega^{N_1N_2N_3n_4}\left(\omega^{2k_4}\right)^{N_1N_2N_3n_4}\right)\right)\right)$$
$$\left(\omega^{2k_4+1}\right)^{N_1N_2n_3+N_1n_2+n_1}\left(\omega^{2k_3}\right)^{N_4N_2N_1n_3}$$
$$\left(\omega^{2N_4k_3}\right)^{N_1N_2n_3+N_1n_2+n_1}\left(\omega^{2k_2}\right)^{N_4N_3N_1n_3}\left(\omega^{2N_4N_3k_2}\right)^{N_1n_2+n_1}\left(\omega^{2k_1}\right)^{N_4N_3N_2n_1}$$

where $X_{N_4N_3N_2k_1+N_4N_3k_2+N_4k_3+k_4}$ is an output factor of the $(N_4N_3N_2k_1+N_4N_3k_2+N_4k_3+k_4)$-th multi-dimensional fast number theoretic transform, $x_{N_4N_3N_2k_1+N_4N_3k_2+N_4k_3+k_4}$ is the $(N_4N_3N_2k_1+N_4N_3k_2+N_4k_3+k_4)$-th input matrix, ω is a twiddle factor, $N_1 \cdot N_2 \cdot N_3 \cdot N_4$ is N^4, and N is a degree of the polynomial.

18. The method related to encryption according to claim **17**, wherein the computing circuits comprise at least one first twiddle circuit, at least one stage conversion circuit, at least one transposition circuit, and at least one memory transposition circuit, the method comprising:

performing a multiplication computation of the twiddle factor through the at least one first twiddle circuit;

performing a butterfly computation through the at least one stage conversion circuit;

performing factor position swapping through the at least one transposition circuit; and

performing memory address change through the at least one memory transposition circuit.

19. The method related to encryption according to claim **12**, wherein:

a mathematical expression of the multi-dimensional fast number theoretic transform is:

$$X_{k_1,k_2,\ldots,k_d} = \sum_{n_1=0}^{N_1-1}\left(\omega_{N_1}^{k_1n_1}\sum_{n_2=0}^{N_2-1}\omega_{N_2}^{k_2n_2}\cdots\sum_{n_d=0}^{N_d-1}\omega_{N_d}^{k_dn_d}\cdot x_{n_1,n_2,\ldots,n_d}\right)$$

where $X_{k_1,k_2,\ldots,k_d}$ is an output factor of $(k_1, k_2, \ldots, k_d)$-th multi-dimensional fast number theoretic transform, $x_{n_1,n_2,\ldots,n_d}$ is $(n_1, n_2, \ldots, n_d)$-th input matrix, ω is a twiddle factor, $N_1 \cdot N_2 \cdot \ldots \cdot N_d$ is N^d, and N is a degree of the polynomial.

20. The method related to encryption according to claim **12**, wherein the computing circuits comprise a stage conversion circuit and a memory circuit, the method further comprising:

performing a butterfly computation through the stage conversion circuit; and

storing data to at least one memory through the memory circuit.

* * * * *